CSE 5324-001 — Project Progress Report Template

Agile Object-Oriented Software Engineering Project | Submission 2

Project Title	PawConnect - Animal Shelter & Adoption Management System
Team Members	Harsha Prasad - 1002230012 Himanshu Shimpi - 1002304702 Dipanjana Das - 1002327740
Class Number	CSE 5324-001
Semester	Fall 2026
Instructor	Farnaz Farahanipad
How to use this template: Replace all bracketed placeholders with project-specific content. Keep the section headings and organize the report in the order shown. Delete instructional guidance before final submission unless the instructor asks you to retain it.	

1. Executive Progress Summary

We are about eight weeks in and have moved from planning into building. The team has a GitHub repo that uses branches and pull requests, a Jira board for the backlog, a React + TypeScript frontend prototype, the Node.js/Express backend set up, and a MySQL 8 database with the seven-table schema and ERD. The use cases, sequence diagrams and class diagrams are done, and we are using them as the reference for the remaining APIs.

What works now. The frontend runs locally and walks through the whole adoption journey using sample data. A visitor can browse and filter animals, open a profile, register or sign in, submit an application and track it. On the admin side, we can review applications, manage appointments, complete an adoption and view the history. The backend server and database are ready for the feature work. What is left is connecting the frontend to the REST API, finishing the server-side business rules and writing the automated integration tests.

What is still in progress. The Express structure, configuration, database connection, schema and persistence design are finished. The feature APIs are not all written yet, and the frontend is not fully connected to the backend. Still being wired up and tested are saving data end to end, JWT login across the UI and API, role checks on the server, the application workflow rules, and appointment conflict handling.

Changes from the proposal. The scope is the same: visitors, registered adopters, one administrator, dogs and cats, and no payments or foster-family management. The changes are mostly tooling and detail. We used Jira for the backlog instead of GitHub Projects, the frontend ended up on React 19, TypeScript and Tailwind CSS 4, and we added three use cases while modeling (UC-06 to UC-08). The status names for appointments and applications also differ between the prototype and the schema, and we still need to line them up (see Section 2.2).

Are we on track? Mostly yes. The frontend, backend and database setup, and modeling are in place, so the main work now is finishing the feature APIs and connecting the frontend to them. We can make the final submission if the next sprint goes to connecting the frontend to the API and database, finishing the business rules, and running automated API and end-to-end tests.

Main user flow: Browse Animals > View Profile > Register/Login > Submit Application > Admin Review > Appointment > Complete Adoption > Adoption History

2. Current Product Scope and Changes Since Proposal

2.1 Current Scope

- Public browsing of dog and cat listings with search by name or breed and filters for type, status, and gender; detailed animal profiles with health information and medical history.
- Adopter accounts: register, log in/out, submit an adoption application, view own applications, schedule a meet-and-greet after approval, view own appointments and adoption history.
- One administrator: manage animal records and statuses, medical records, review (approve/reject) applications, manage appointments, complete adoptions, review adoption history.
- Business rules: no applications for Adopted animals, no conflicting appointments, valid animal status transitions, role-based access, hashed passwords.
- Still out of scope: payments, donations, billing, multiple branches or admin levels, mobile apps, AI recommendations, live chat, external integrations, foster-family management (Fostered remains only an animal status).

2.2 Scope / Requirement Changes

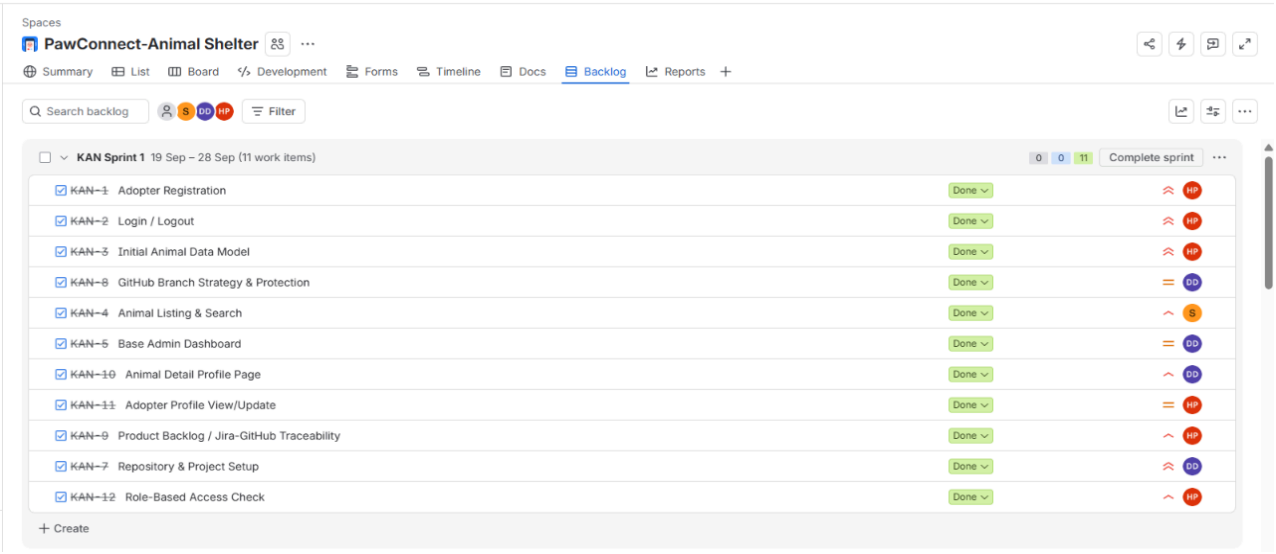
Change ID	Original Item	Change	Reason	Impact
CH-01	Backlog tool: GitHub Projects (proposal Section 4)	Changed to a Jira board (project KAN); GitHub is used for code, branches and pull requests only	Team preferred Jira boards for sprint planning and reporting	Process only; no code impact
CH-02	Frontend: React + Bootstrap + CSS	Changed to React 19 + TypeScript + Vite + Tailwind CSS 4	Type safety for shared data shapes (Animal, Application, Appointment); faster UI iteration	Design / code; no requirement change
CH-03	Backend/DB: "SQL" database, stack open	Fixed to Node.js + Express with request validation, MySQL 8, Knex migrations, Vitest + Supertest	Stack decision required by risk plan (lock stack early)	Design; schema now in PawConnect_MySQL_Schema.sql
CH-04	FR-05 search by type, breed, age, status	Clarified: prototype searches by name/breed text and filters type, status, gender; age filter not yet built	Gender filter was listed in proposal Section 3 but not in FR-05; age filter deferred	Code; FR-05 stays Partial
CH-05	Application statuses: Pending, Approved, Rejected	Added Withdrawn (adopter can withdraw) in the prototype; schema currently has a Completed status instead	Adopter-facing convenience discovered during UI design	Schema / code: align both lists (see BUG-05)
CH-06	FR-17/19: appointment Requested > Confirmed/Completed/Cancelled	Prototype books a Scheduled appointment tied to an Approved application and completes the adoption from a completed appointment	Simpler flow for a single-admin shelter	Design / schema / code: decide final statuses in Sprint 3
CH-07	Use cases UC-01 to UC-05	Added UC-06 Register/Log In/Log Out, UC-07 Manage Animals and Medical Records, UC-08 Manage Adopter Profile	Requirements FR-01 to FR-03 and FR-07 to FR-09 had no business-level use case	Design (Section 6)
CH-08	Adopter views own history (implied by FR-21)	Added a "My Adoption History" page for adopters	Adopters need visibility of completed adoptions	Code (done in prototype)

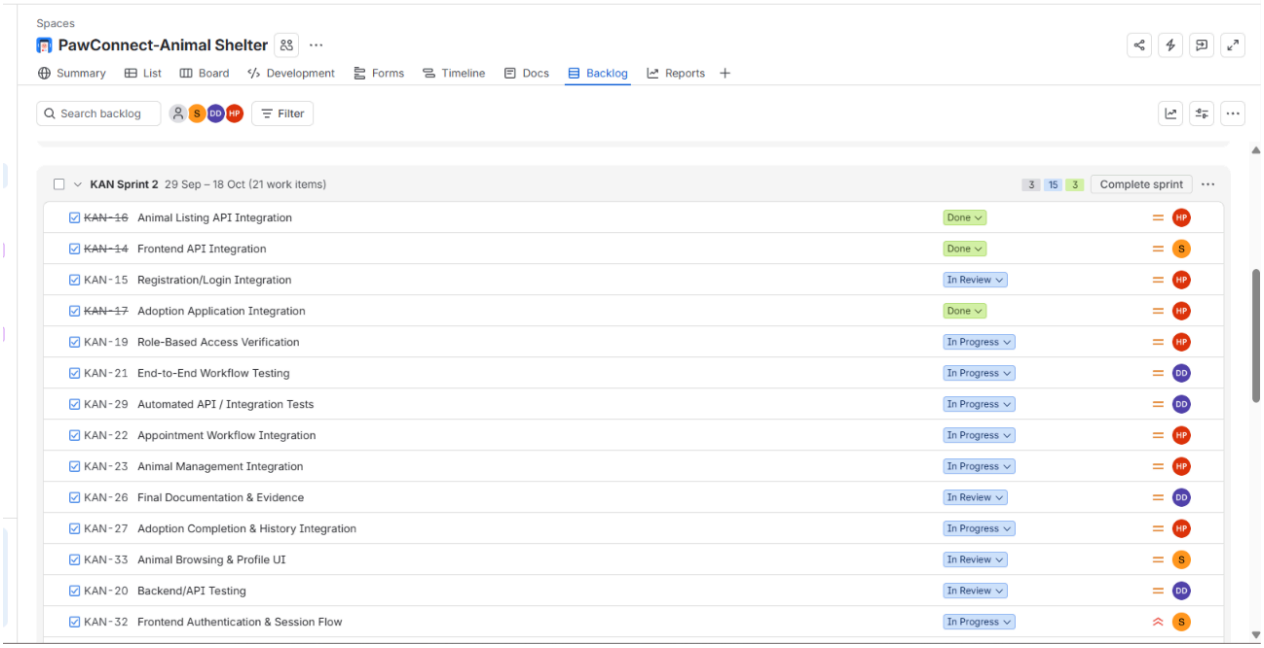
3. Agile Iteration / Sprint Summary

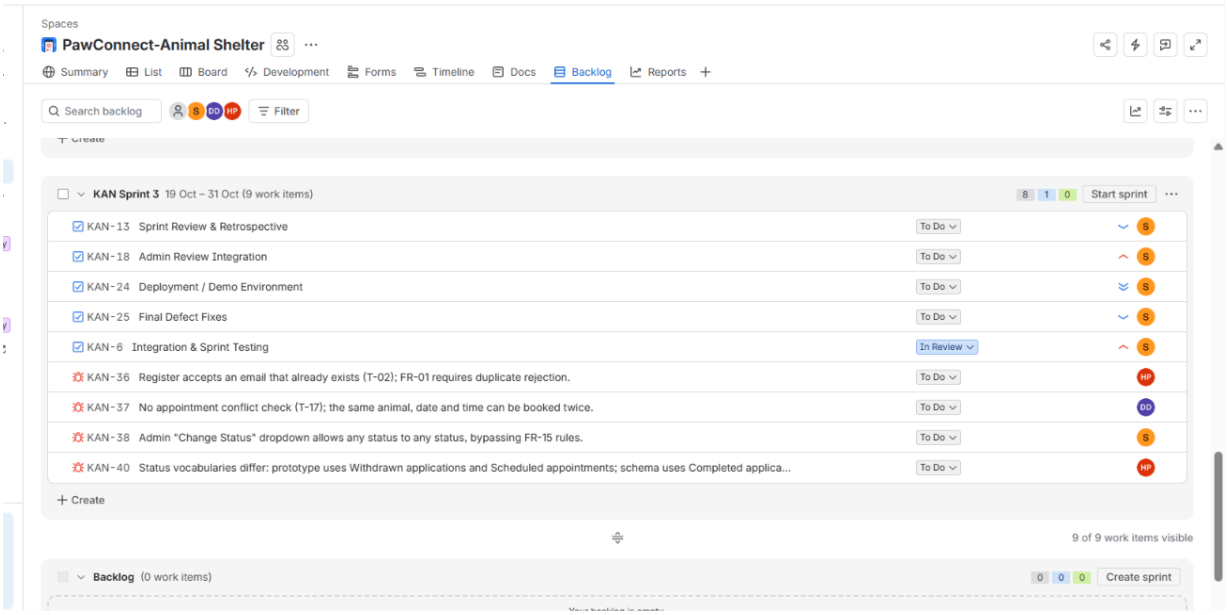
Sprint	Goal	Stories Planned	Stories Completed	Demo / Review Result	Retrospective Action
Sprint 1 [TEAM: confirm dates, e.g. Sep 21 - Oct 4]	Project setup and public/adopter UI foundation	US-01, US-02, US-03; repo setup; board setup	Repo, branches and Jira board; UI for US-01 (register/login forms), US-02 (listing, search, filters), US-03 (profile with health data). Delivered on sample data.	Frontend merged to main by pull request #1 on Oct 6 and runs locally	Start API and DB work in parallel with UI so stories are done end to end; keep README status accurate
Sprint 2 [TEAM: confirm dates, e.g. Oct 5 - Oct 18]	Admin management and the core application workflow	US-04, US-05, US-06	UI for US-04, US-05 and US-06; backend project setup and MySQL database setup completed; schema/ERD in place. Feature APIs and frontend integration remain in progress.	Frontend workflow demonstrated; backend server and database foundation verified separately. End-to-end API integration is still in development.	Complete feature APIs and connect the React frontend to the backend/database early in Sprint 3; add automated API tests.
Sprint 3 (planned)	Complete API features and integrate frontend with backend/database; finish appointments and adoption workflow	US-01 to US-07 integration	Backend/database setup complete; feature integration in progress	Integration demo planned	Test each integrated story against its acceptance criteria and keep Jira/GitHub evidence current
Sprint 4 (added)	Hardening, testing, deployment, final documentation	Defect fixes, tests	Not started	n/a	n/a

3.1 Agile Evidence

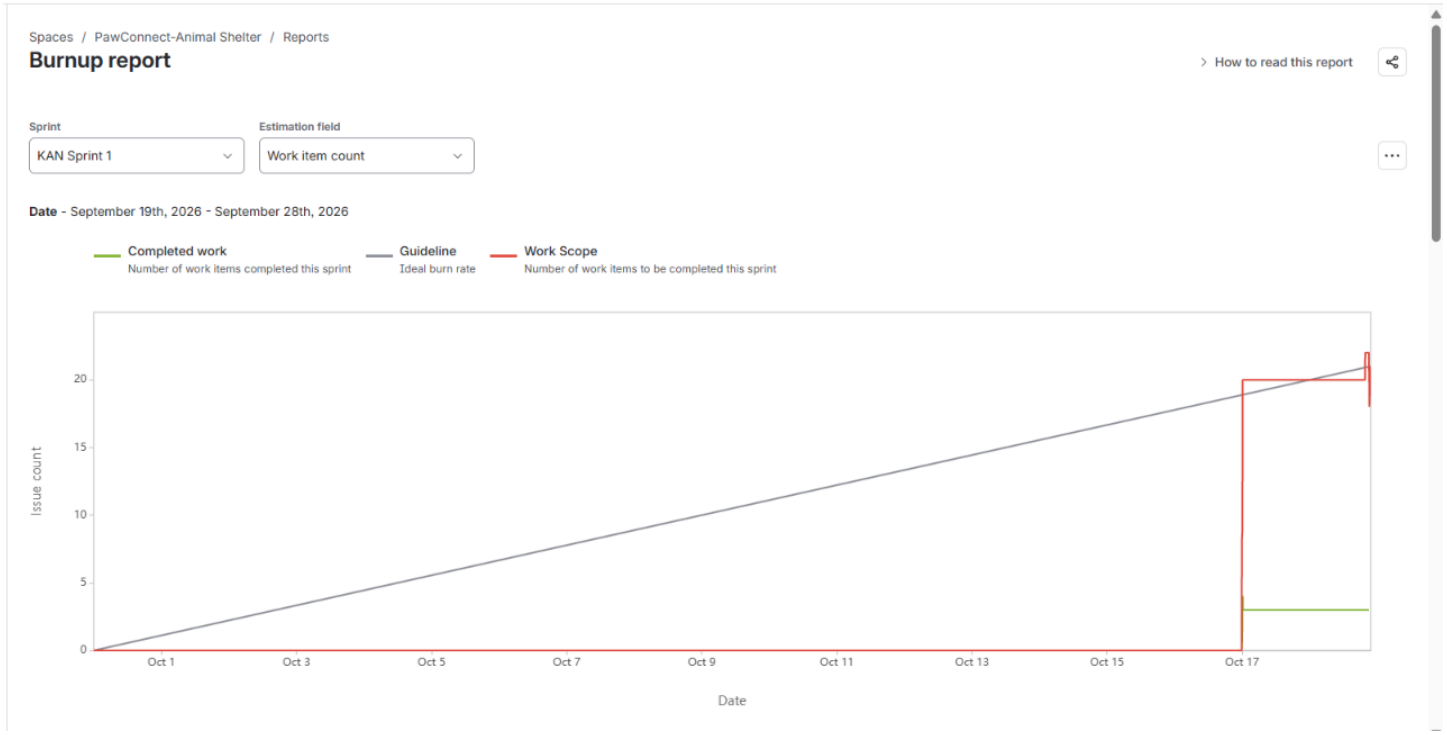
- screenshot/export of the current product backlog or project board.

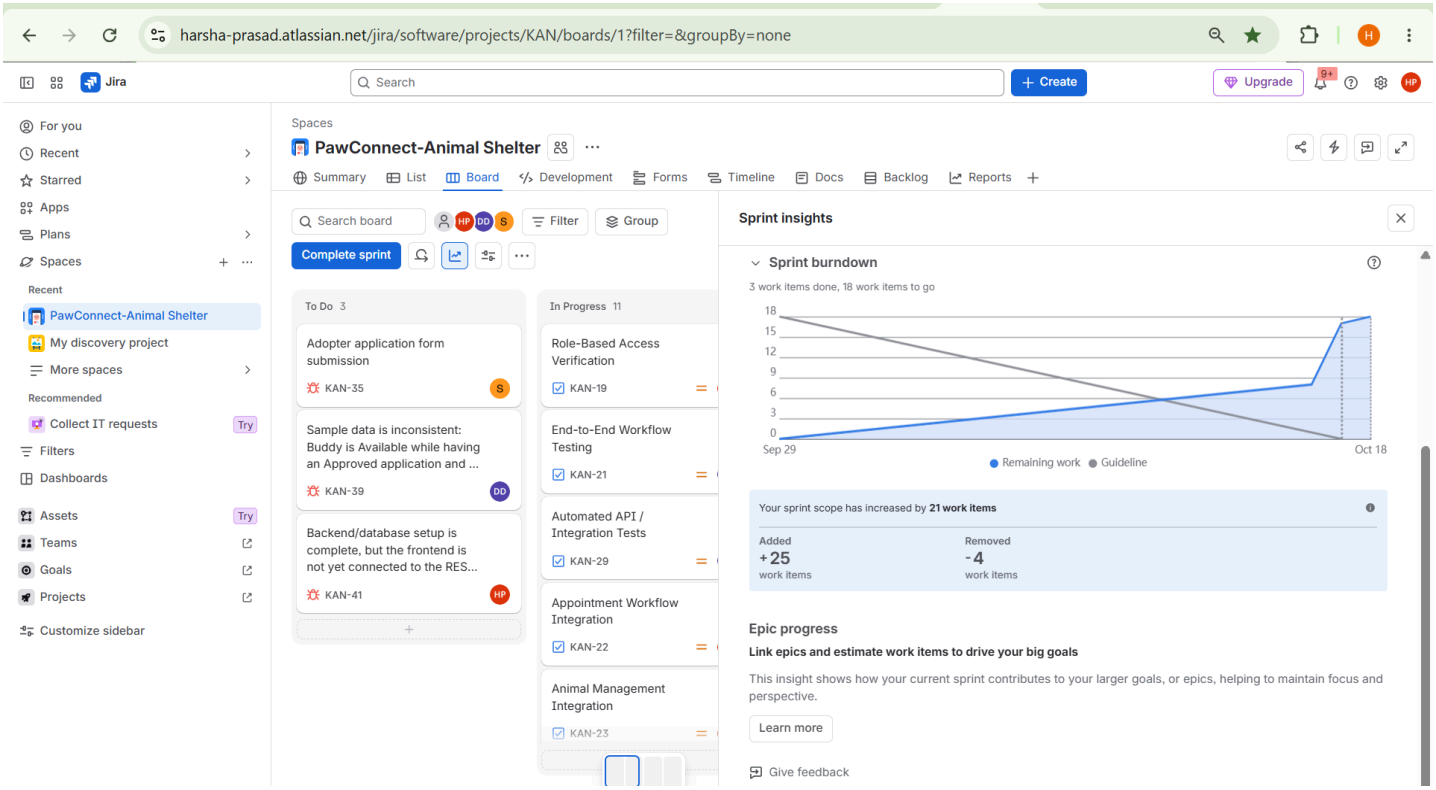






- simple burndown, burnup, cumulative-flow, or completed-story chart if available.

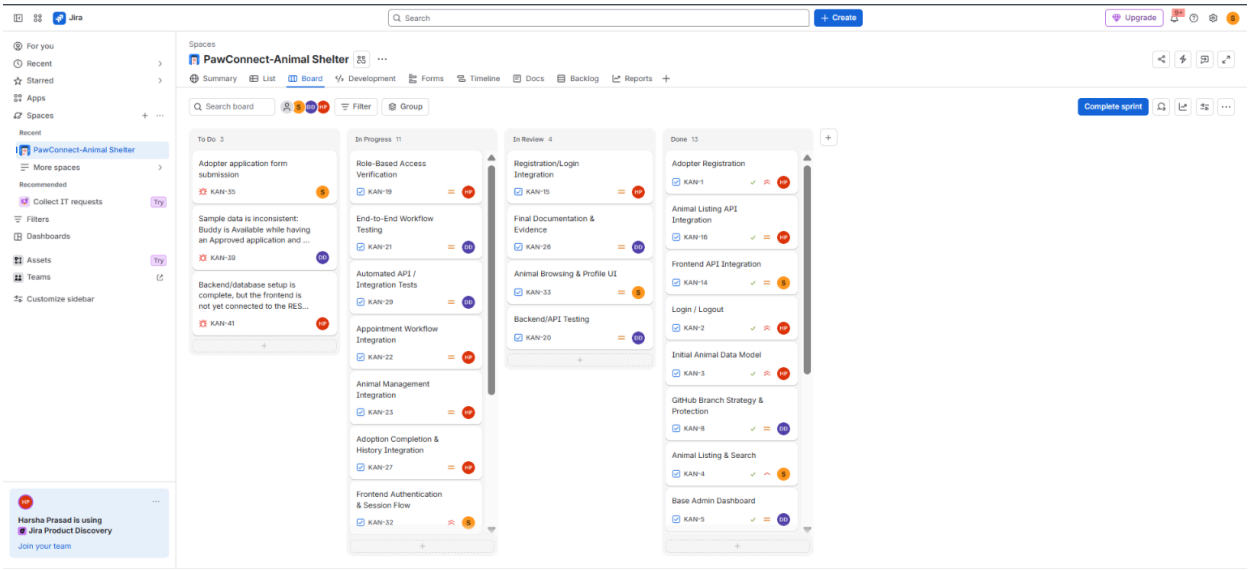


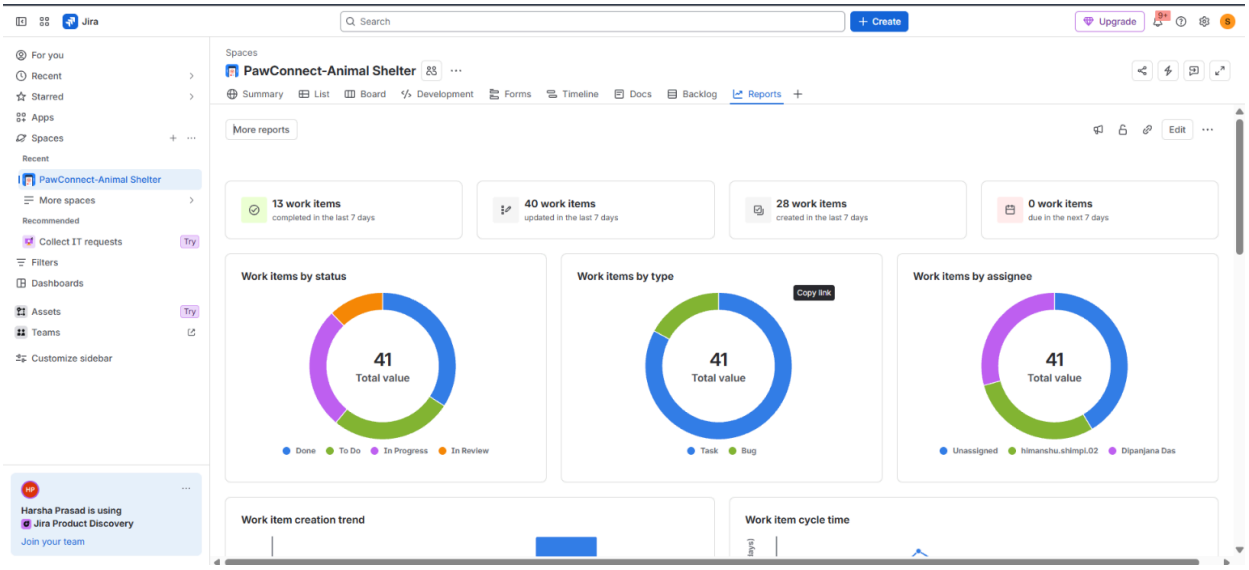


Velocity and trend. Sprint 1 and Sprint 2 delivered the main frontend workflows and established the backend/database foundation. The backend server setup and MySQL schema are complete, but most stories are still marked Partial because the frontend and REST API are not yet connected end to end. The team therefore treats infrastructure/setup tasks as complete while feature stories remain in development until persistence, authorization, and acceptance tests pass. Jira velocity numbers should replace this qualitative statement once exported.

Backlog (from the proposal, unchanged): US-01 registration/login; US-02 browse and filter; US-03 animal profile; US-04 admin

animal and medical records; US-05 submit and track application; US-06 admin review; US-07 appointments, adoption completion and history.





4. Working Product Increment - What Is Completed

The current increment includes a runnable React prototype plus a completed backend/database foundation. The UI workflows currently run on sample data, while the Express server structure, database connection, MySQL schema, and persistence model are set up. Frontend-to-API integration and feature-level backend verification are still in progress, so no feature should be considered fully complete end to end until that integration is tested.

Feature / User Story	Related Use Case	Status	Evidence	Known Limitations
US-01 Register and log in	UC-06	UI done	Figures 4.1-4.2; T-01, T-03, T-04, T-05	Duplicate email accepted (BUG-01); no persistence; no JWT yet
US-02 Browse, search, filter	UC-01	UI done	Figure 4.3; T-06, T-07, T-08	No age filter; sample data only (8 animals)
US-03 Animal profile	UC-01	UI done	Figure 4.4; T-09, T-12	Photos are remote placeholder URLs
US-04 Admin animals and medical	UC-07	In progress	Figures 4.9-4.10; pages	Add/update flows not yet covered by tests;

records			exist (add animal, add record)	status dropdown unrestricted (BUG-03)
US-05 Submit and track application	UC-02	UI done	Figures 4.5-4.6; T-10 to T-13	Not saved to DB; confirmation text promises an email (BUG-07)
US-06 Admin review	UC-03	UI done	Figure 4.7; T-14, T-15	Decision note not captured in prototype
US-07 Appointments, completion, history	UC-04, UC-05	In progress	Figures 4.8, 4.11; T-16, T-19	No conflict check (BUG-02); statuses differ from schema (BUG-05)

4.1 Screenshots / Demo Evidence

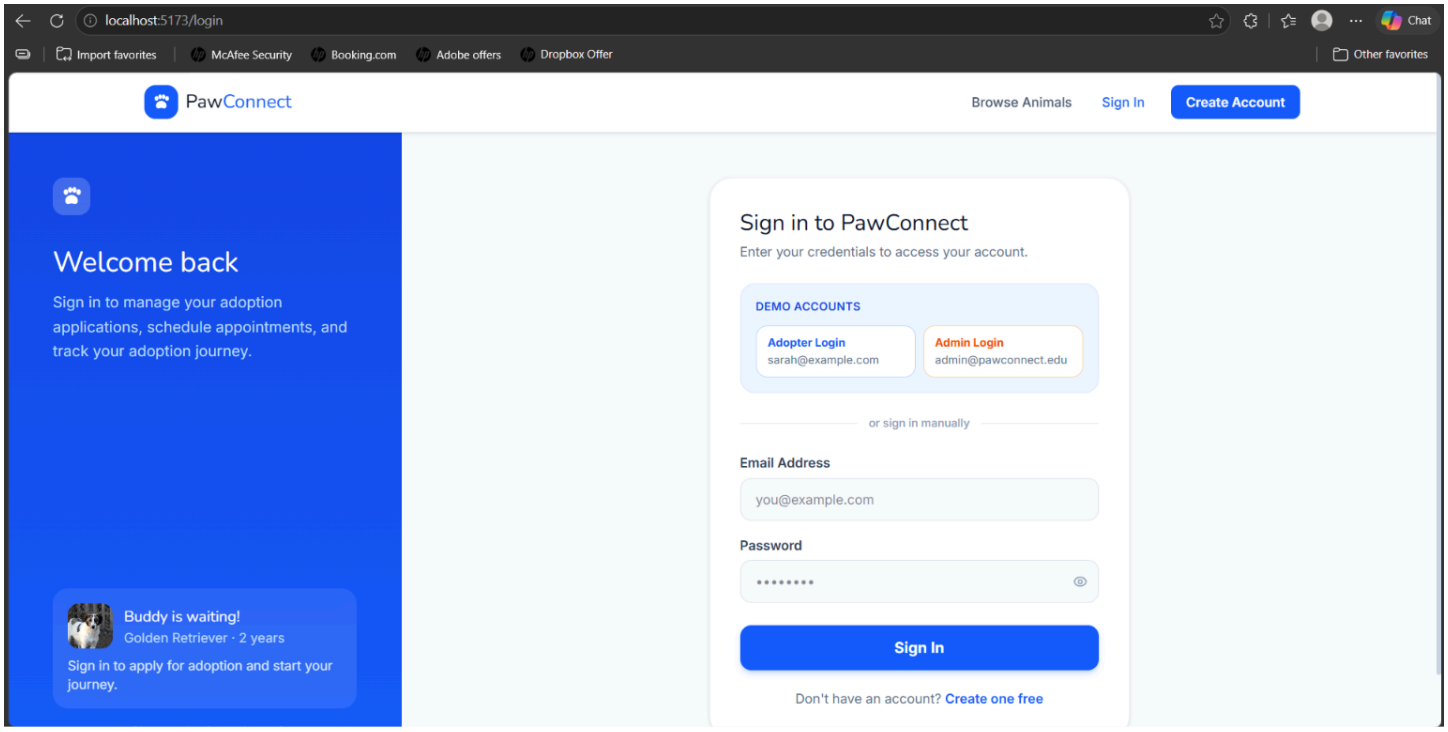


Figure 4.1 - Sign In screen (US-01 / UC-06), with demo accounts

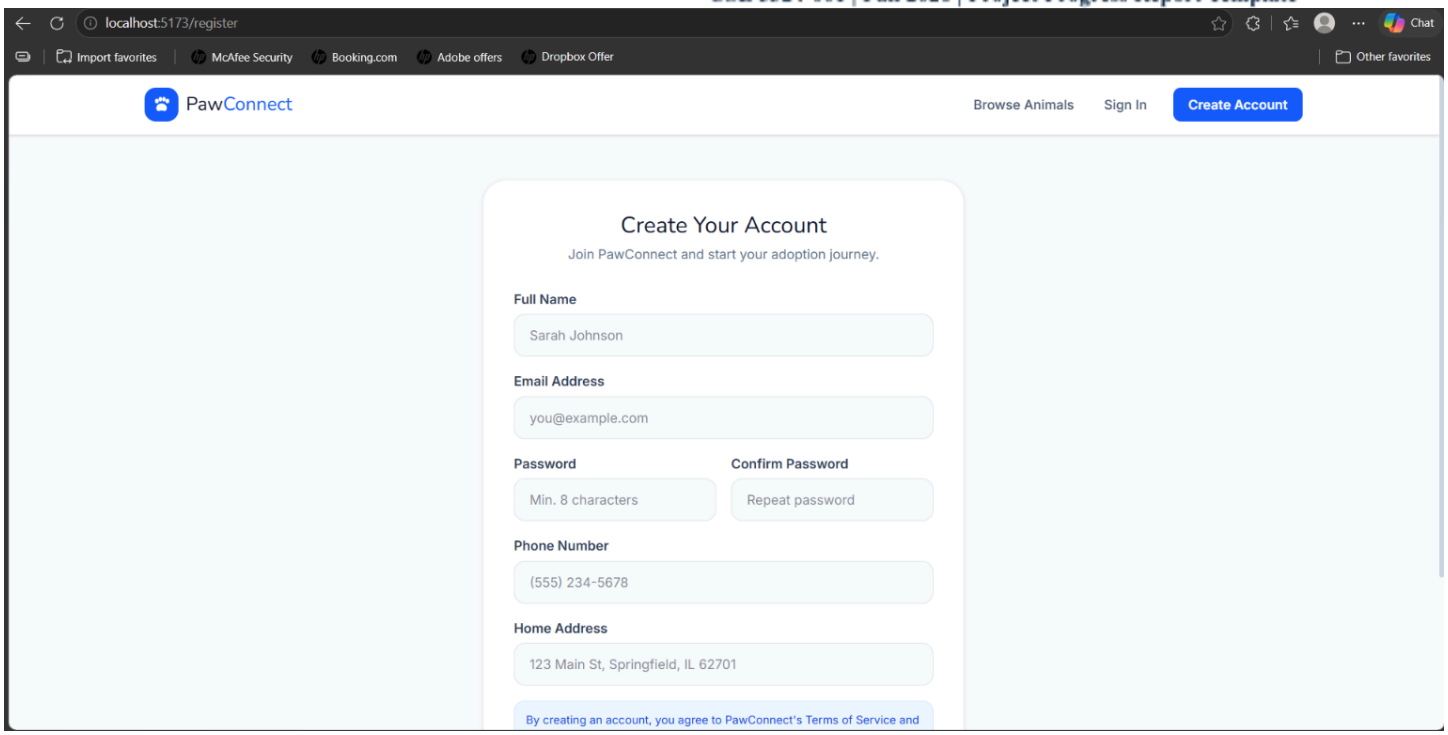


Figure 4.2 - Create Account with field validation (US-01 / UC-06)

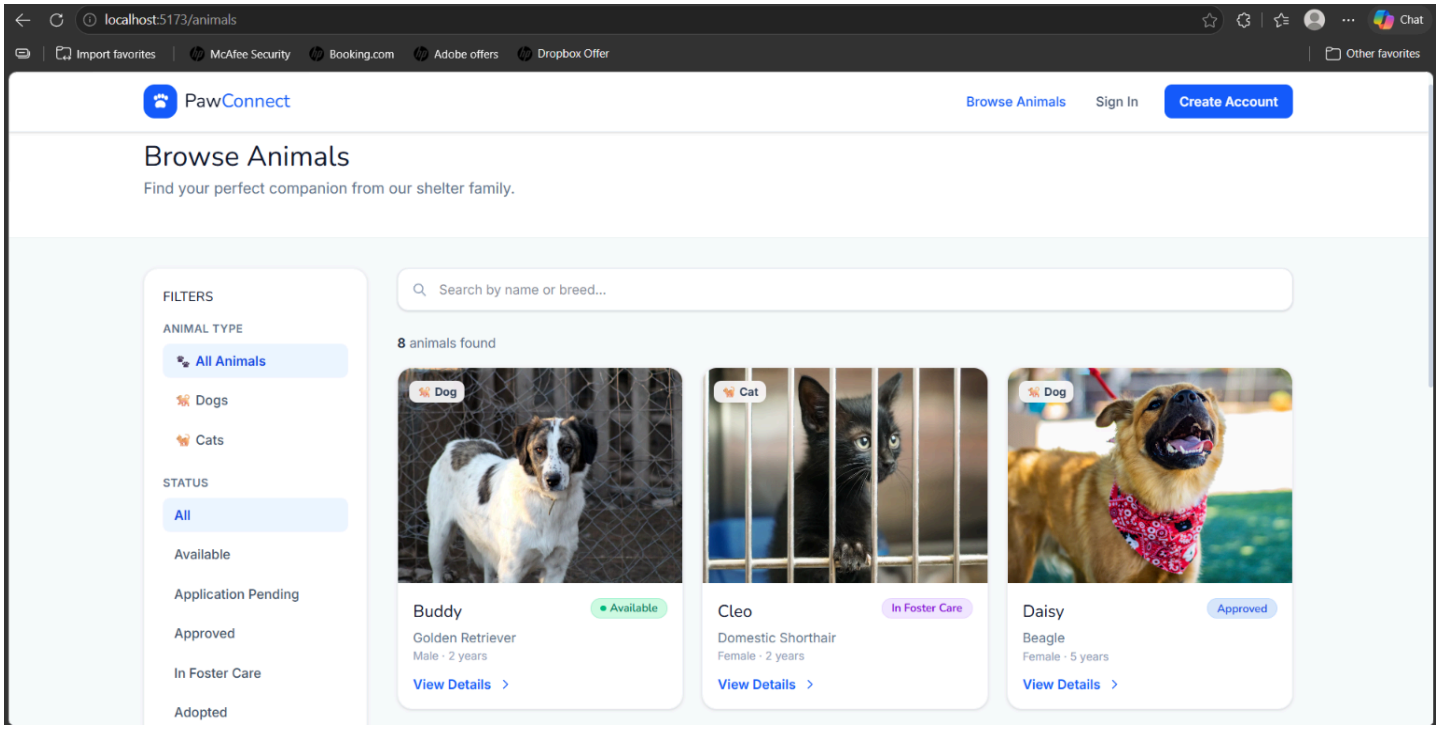


Figure 4.3 - Browse Animals with search and type/status/gender filters (US-02 / UC-01)

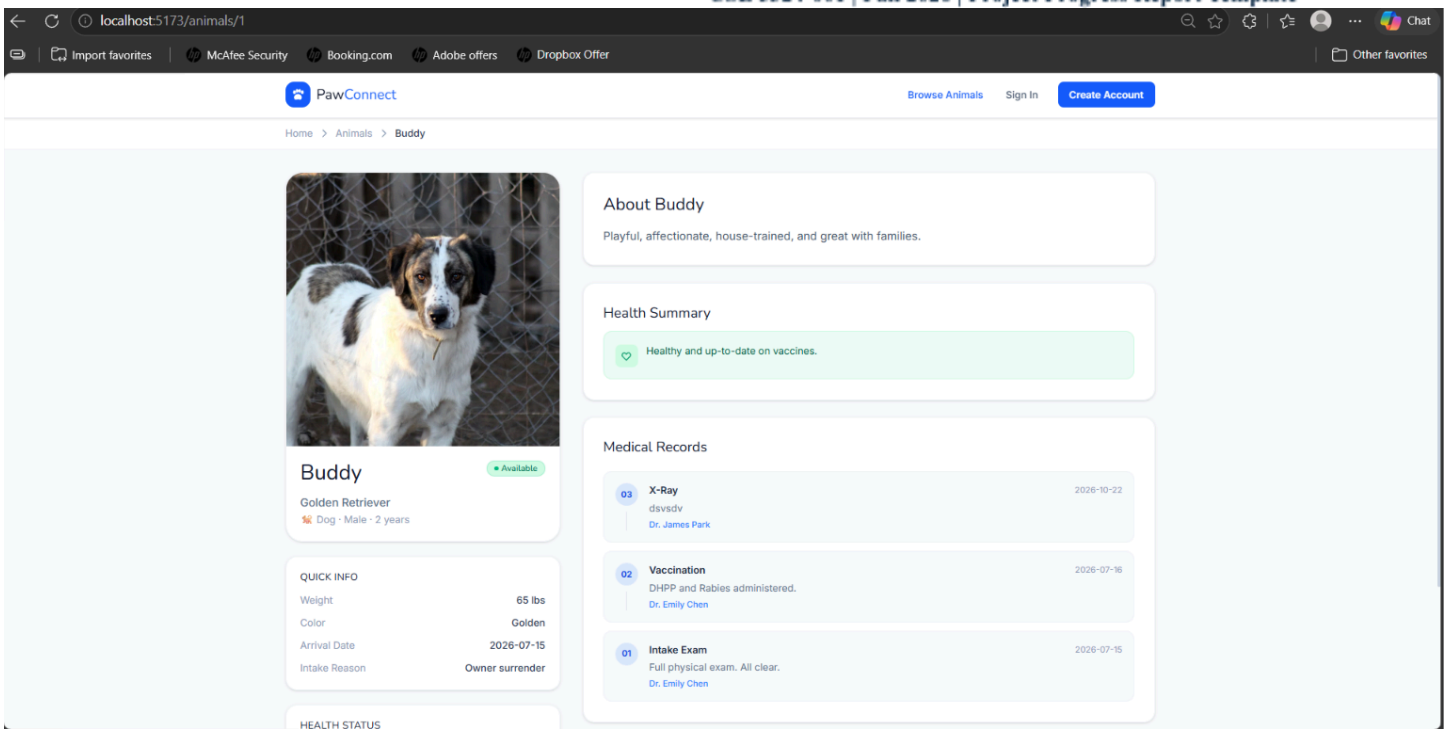
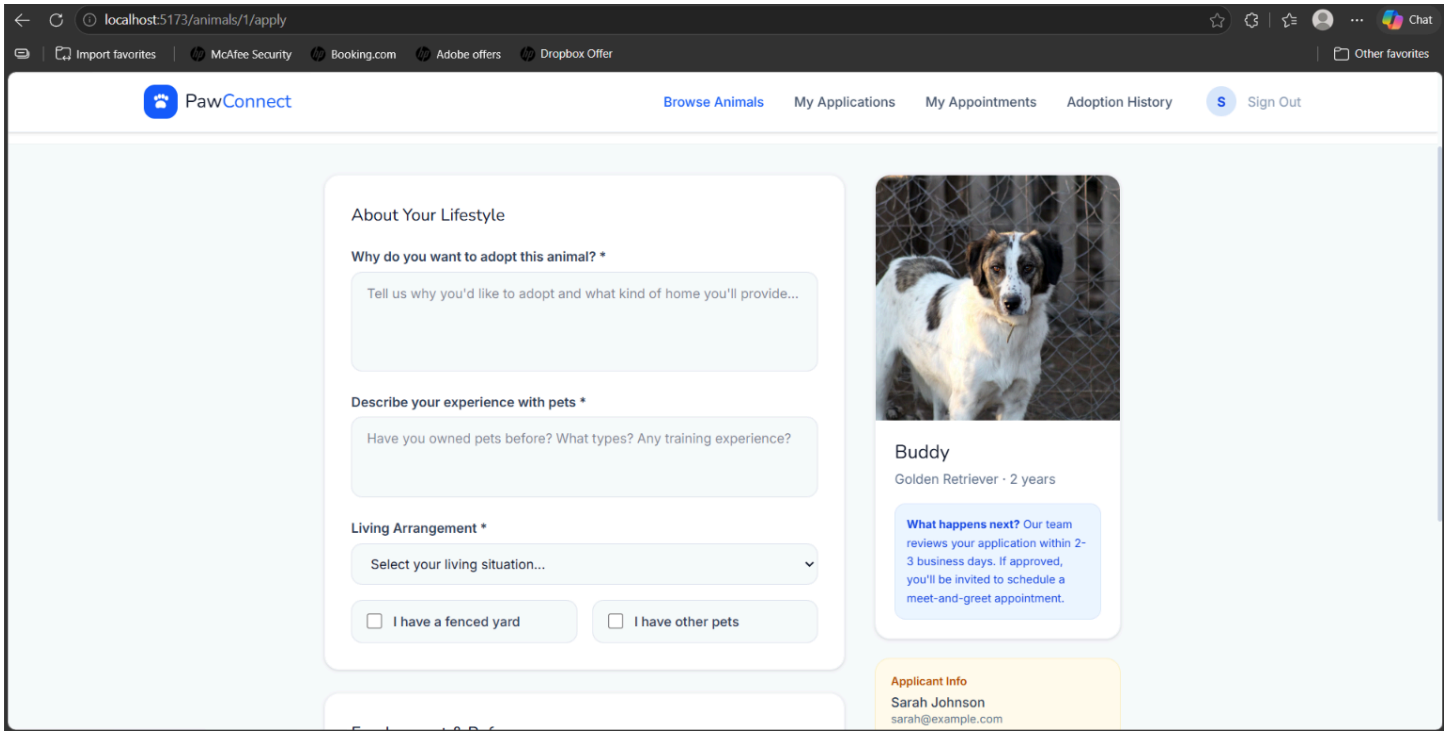


Figure 4.4 - Animal profile with health summary and medical records (US-03 / UC-01)



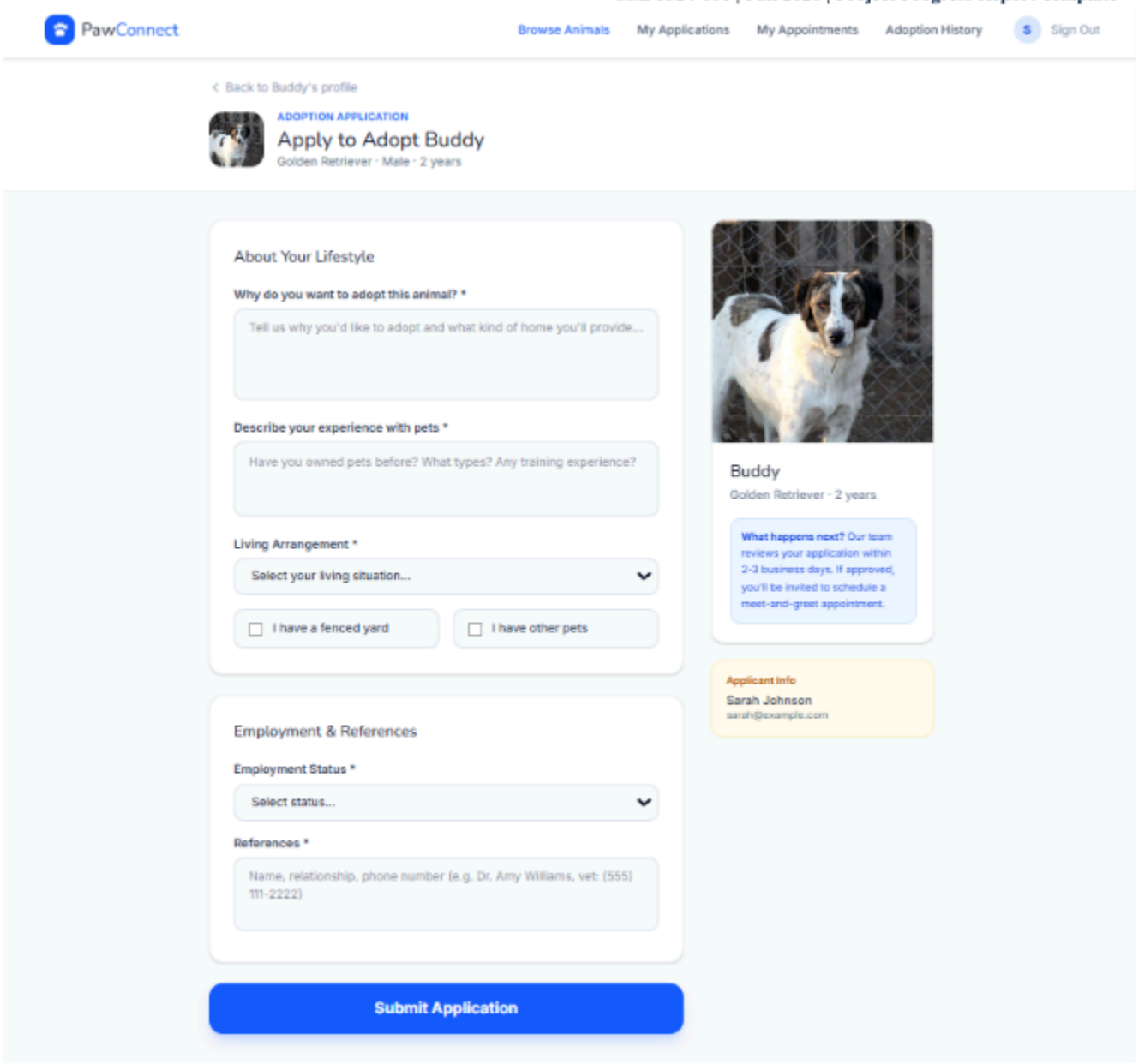


Figure 4.5 - Adoption application form (US-05 / UC-02)

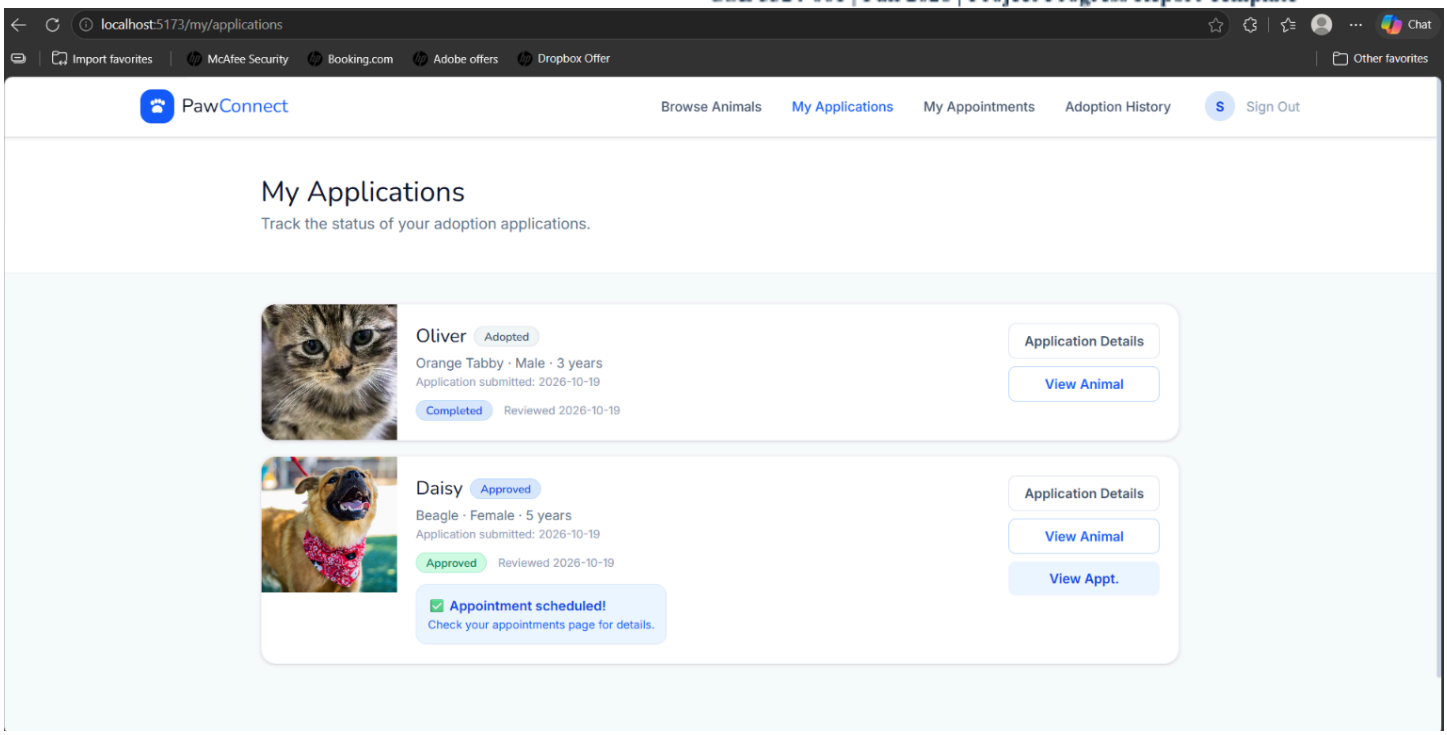


Figure 4.6 - My Applications: adopter sees only own applications (US-05 / UC-02)

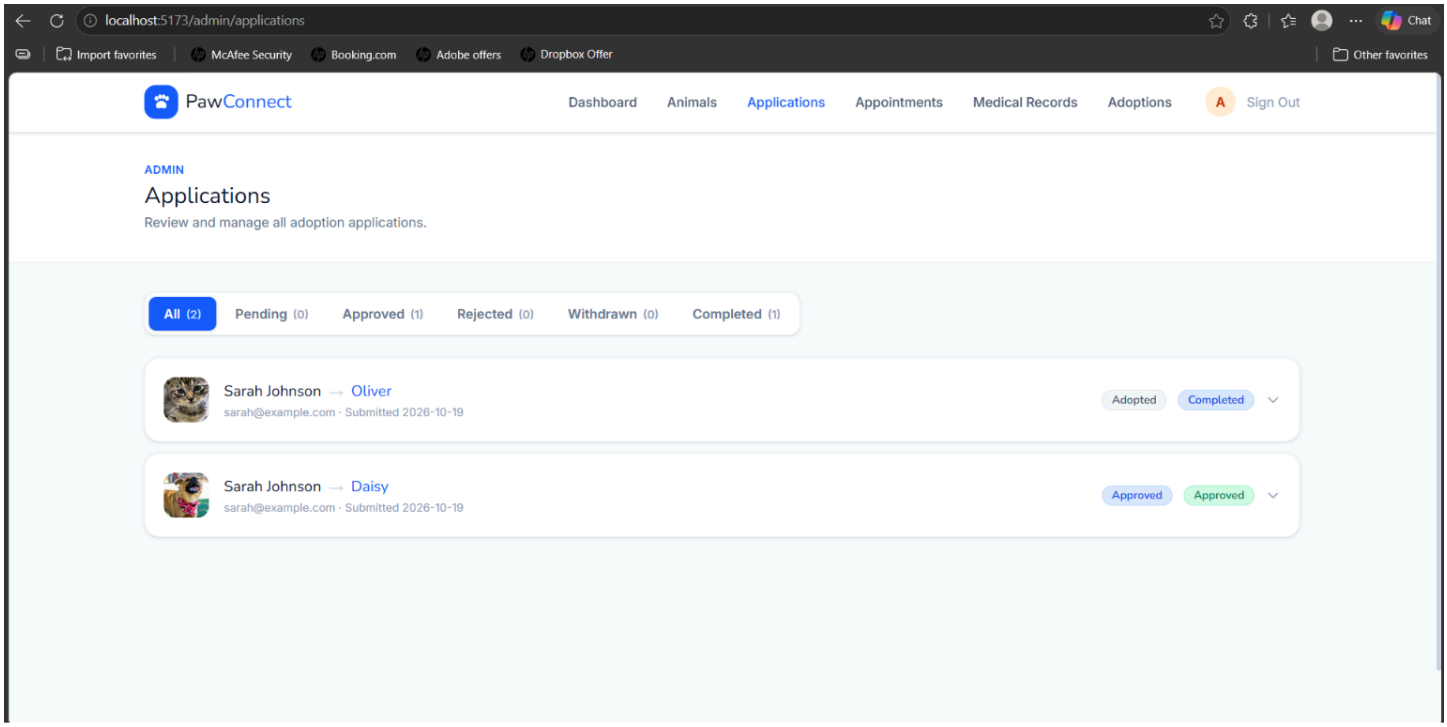


Figure 4.7 - Admin Applications: review, approve or reject (US-06 / UC-03)

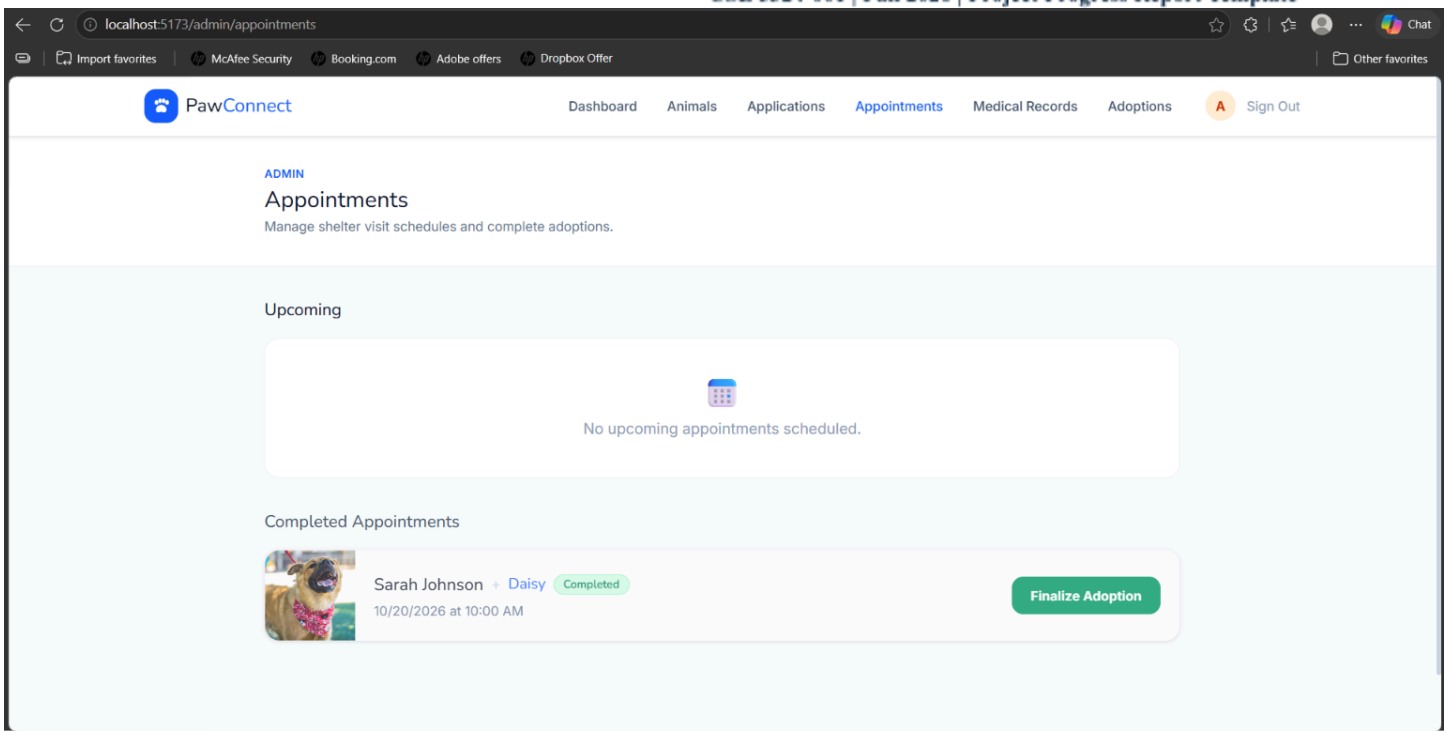


Figure 4.8 - Admin Appointments: mark completed / complete adoption (US-07 / UC-04, UC-05)

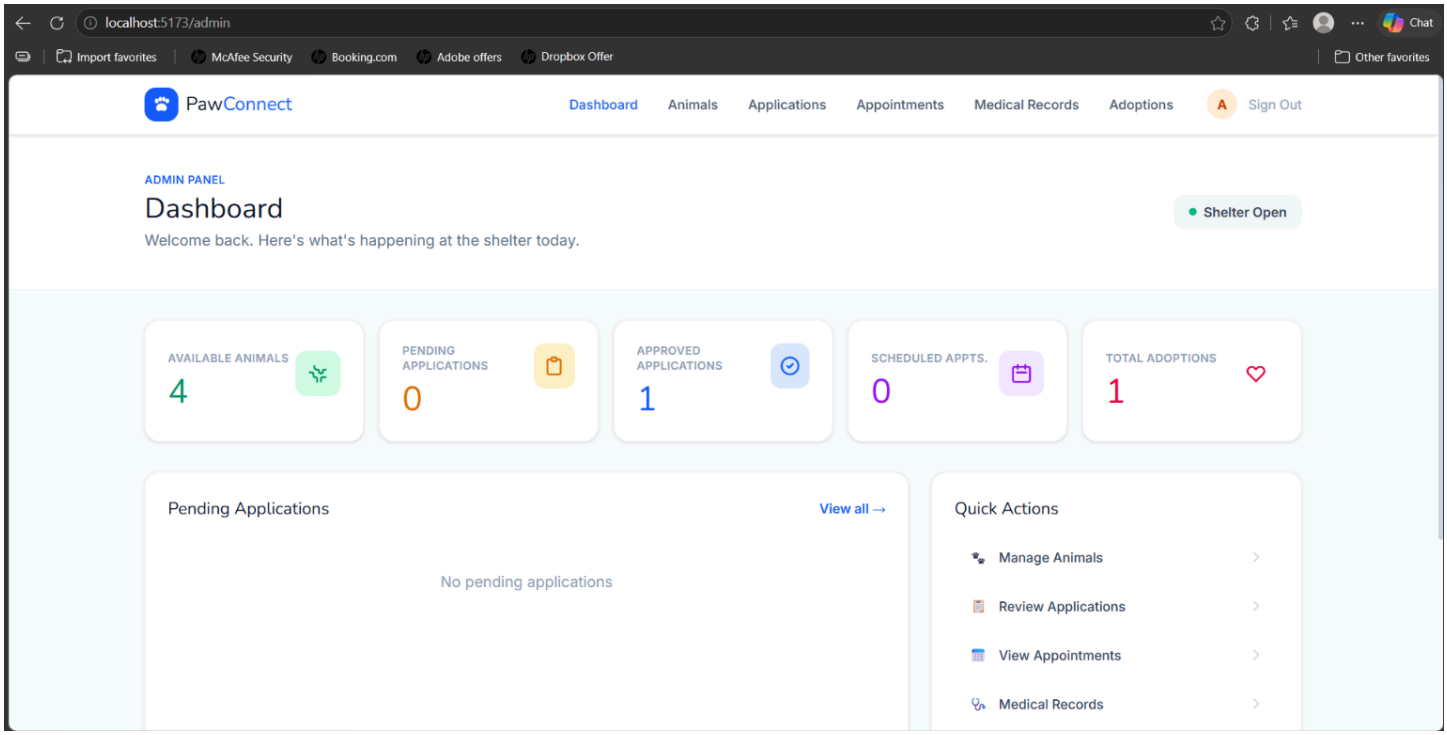


Figure 4.9 - Admin Dashboard summary (US-04 / UC-07)

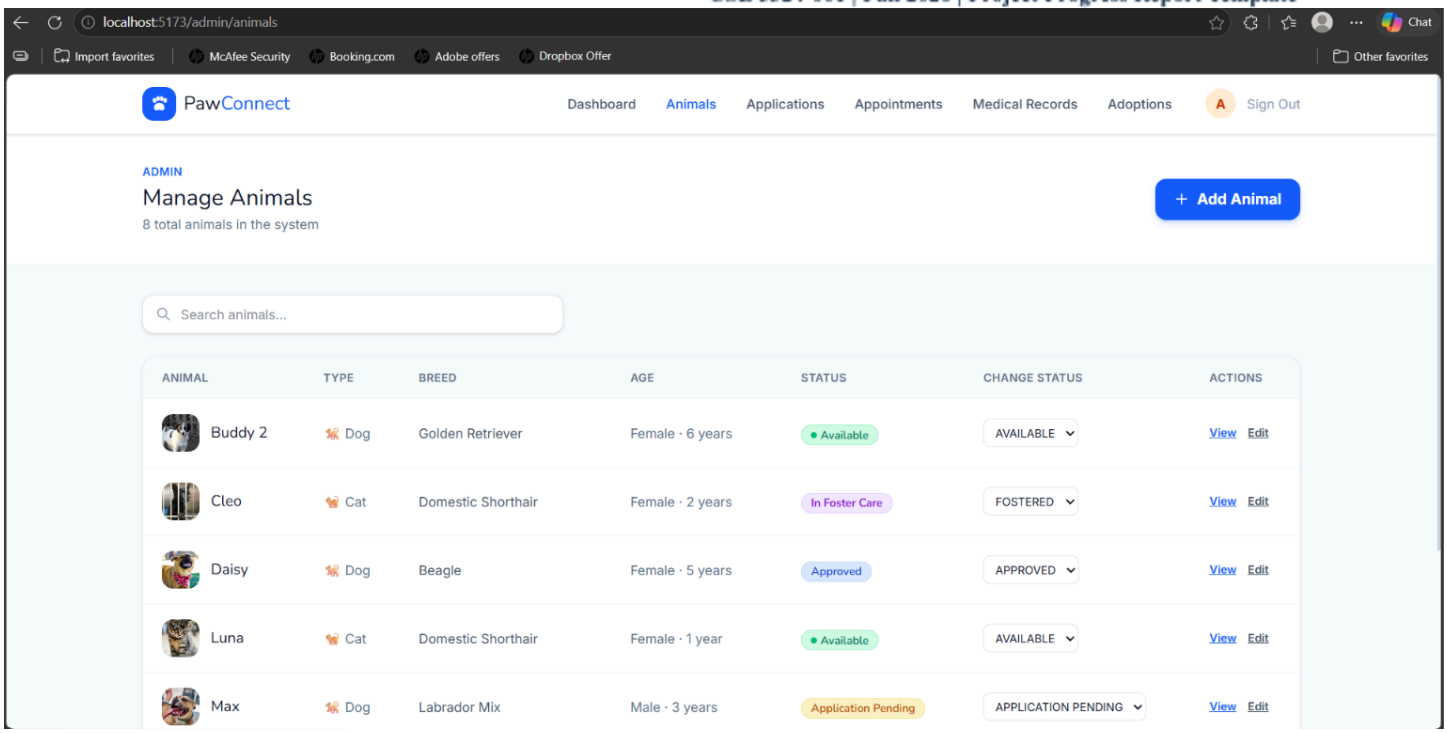


Figure 4.10 - Admin Manage Animals with status control (US-04 / UC-07)

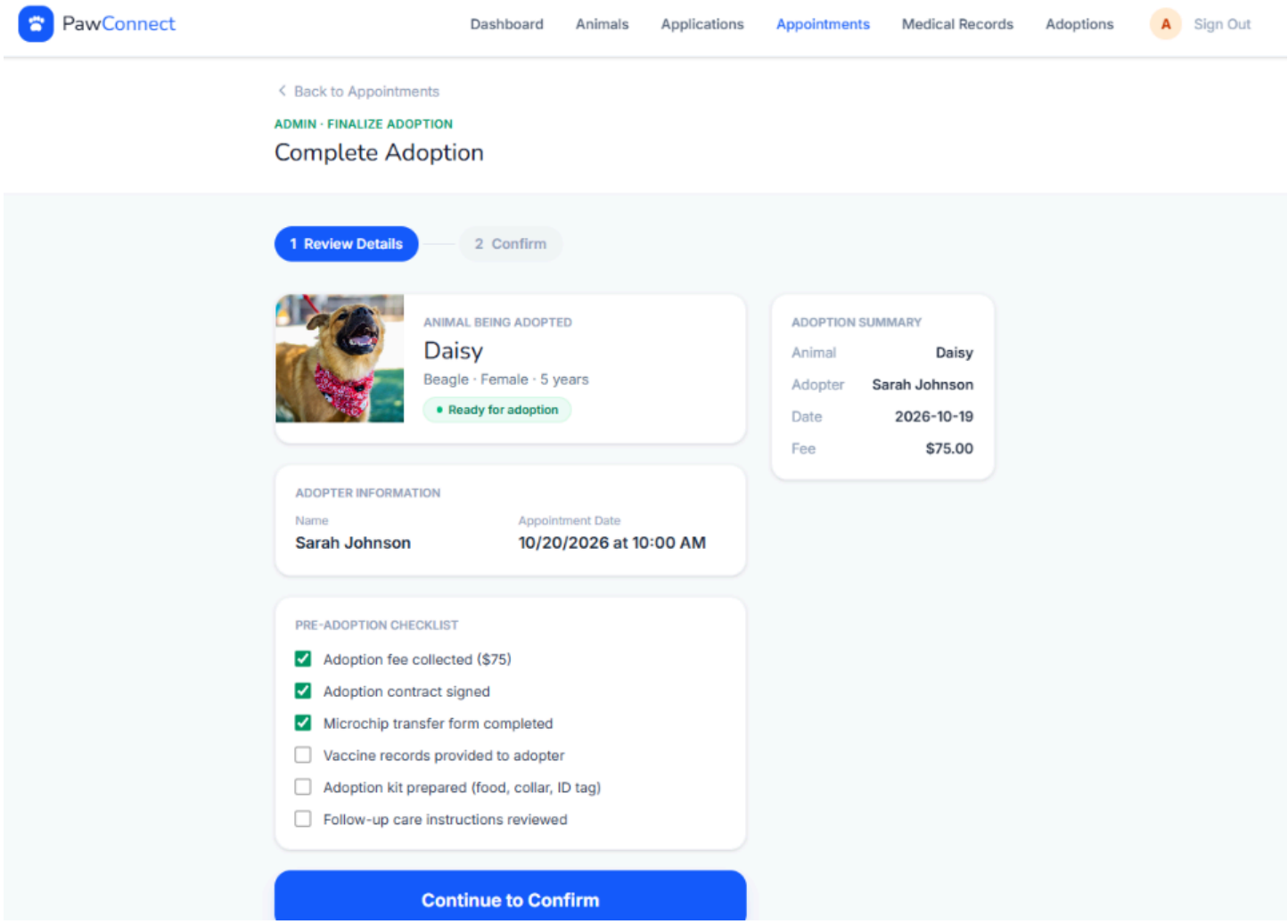


Figure 4.11 - Complete Adoption review step (US-07 / UC-05)

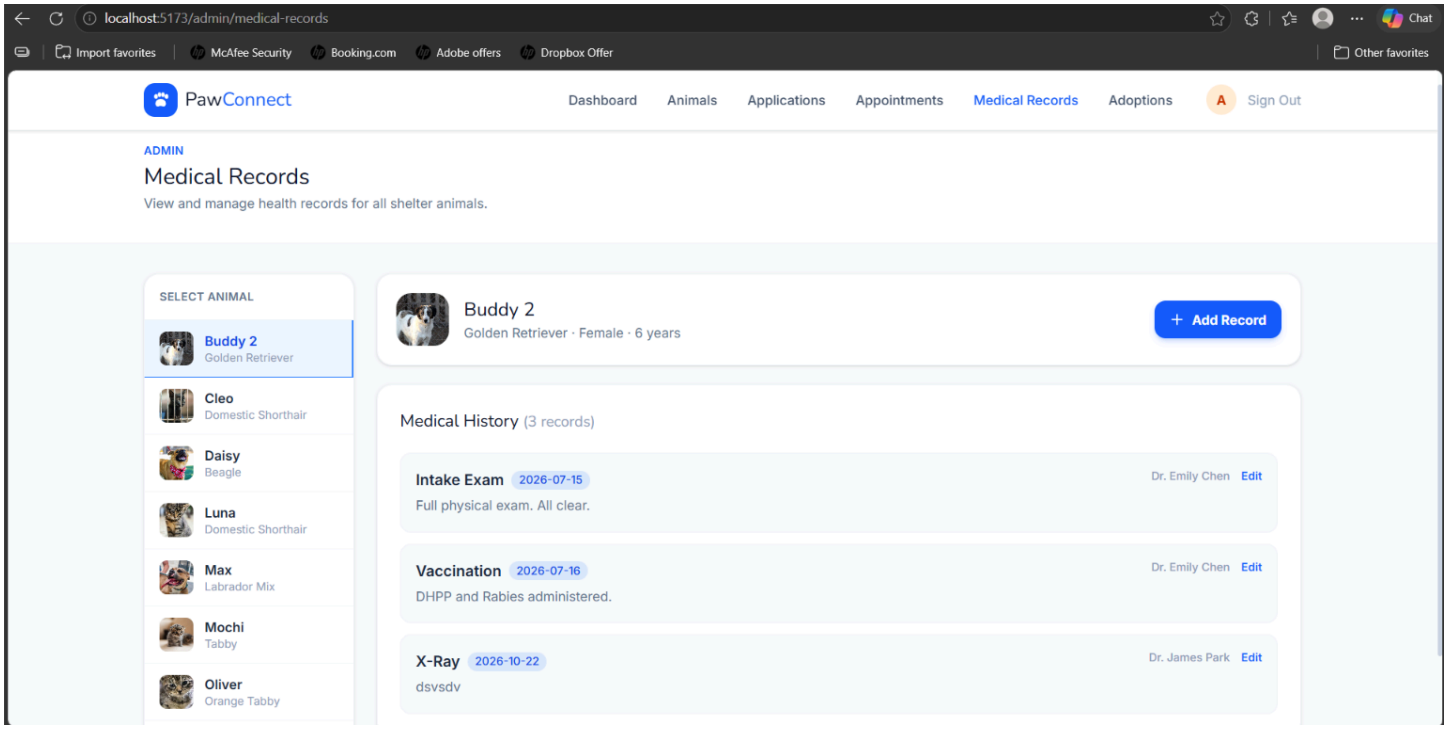


Figure 4.12 - Admin Medical Records (US-04 / UC-07)

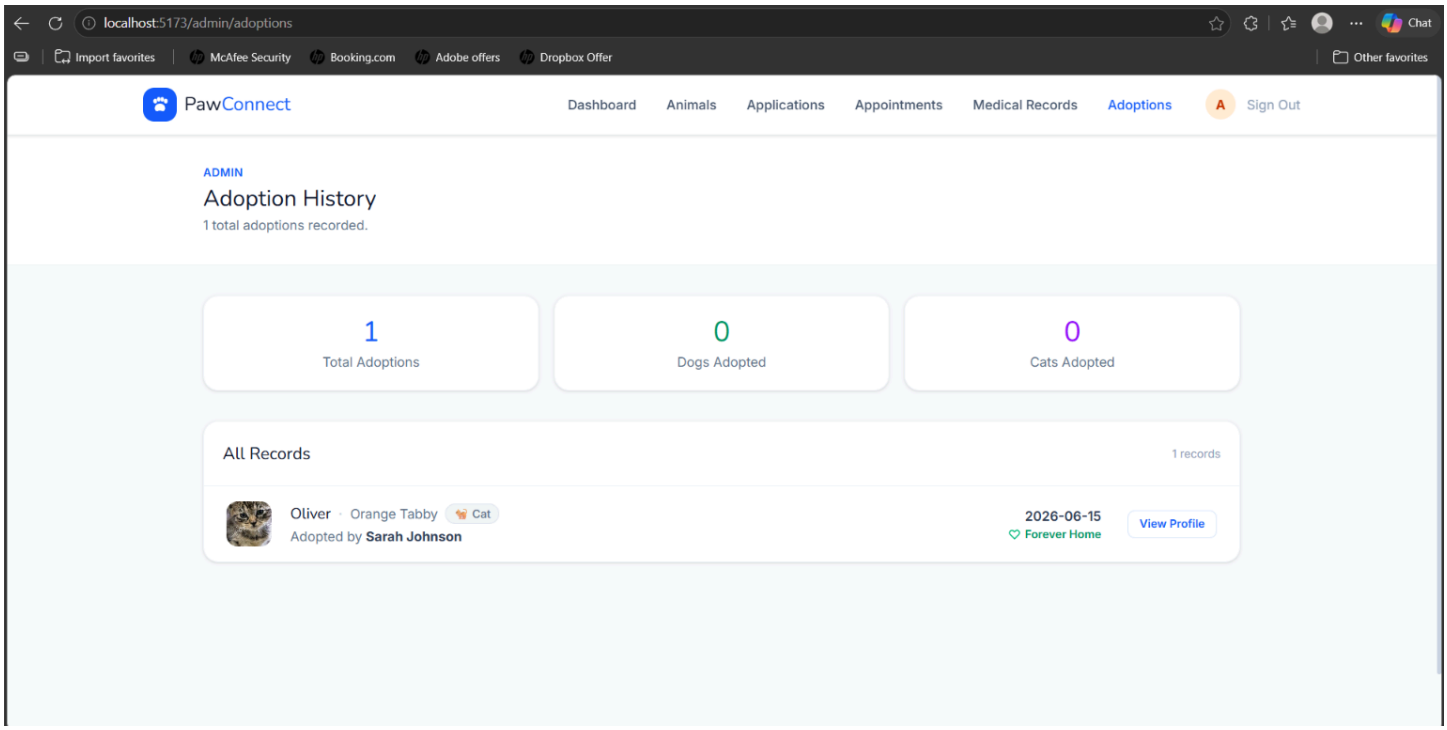


Figure 4.13 - Admin Adoption History (US-07 / UC-05)

5. Updated Requirements / SRS Status

5.1 Functional Requirement Status

ID	Functional Requirement	Status	Implemented By	Evidence / Test
FR-01	The system allows a new adopter to register using basic personal information and a password.	Completed	US-01 / Authentication	UI registration works; backend/database foundation complete; duplicate-email API validation and persistence verification in progress
FR-02	Registered adopters and the Administrator can log in and log out.	Completed	US-01 / Authentication	UI login/logout works
FR-03	A registered adopter can view and update profile/contact information.	Not started	US-01 / Profile module (UC-08)	Backend/database foundation available; profile API/UI workflow still to be implemented
FR-04	Visitors and adopters can browse dog and cat listings without logging in.	Partial	US-02 / Animal listing	UI listing works; database schema ready; live API/database retrieval integration in progress
FR-05	Users can search/filter animals by type, breed, age, and status.	Partial	US-02 / Search and filters	UI search/filter works except age filter; backend query integration still in progress
FR-06	The system displays a detailed animal profile with health information.	Partial	US-03 / Animal profile	UI profile works; live animal/medical retrieval from MySQL still in progress
FR-07	The Administrator can add a new dog or cat record.	Partial	US-04 / Admin animals	Admin form exists; backend/database foundation complete; create-animal API integration in progress
FR-08	The Administrator can update animal profile information and status.	Partial	US-04 / Admin animals	UI status change exists; server-side transition enforcement and persistence integration in progress
FR-09	The Administrator can create, view, and update medical records for an animal.	Partial	US-04 / Medical records	Add/view UI exists; database table ready; CRUD API integration still in progress
FR-10	A logged-in adopter can submit an adoption application for an eligible animal.	Partial	US-05 / Applications	UI submission works; application table ready; server-side persistence and rule

				validation in progress
FR-11	The system prevents an application for an animal that is already adopted.	Partial	US-05 / Application validation	UI blocks adopted animal; server-side validation integration still in progress
FR-12	A registered adopter can view the status of their own applications.	Partial	US-05 / Applications	UI shows own applications; server ownership filtering and DB retrieval integration in progress
FR-13	The Administrator can view all submitted adoption applications.	Partial	US-06 / Admin applications	Admin UI works; live database-backed application retrieval in progress
FR-14	The Administrator can approve or reject an adoption application.	Partial	US-06 / Admin applications	UI approve/reject works; server-side decision persistence and transaction logic in progress
FR-15	The system maintains animal states Available, Application Pending, Approved, Fostered, Adopted.	Partial	Animal status workflow	Status model/schema complete; service-level transition enforcement and status-history persistence in progress
FR-16	The Administrator can view the previous adoption record of an adopted animal.	Partial	US-07 / Adoption history	History UI exists; database-backed per-animal adoption record retrieval still in progress
FR-17	A registered adopter can request an appointment for an eligible animal.	Partial	US-07 / Appointments	Scheduling UI exists; backend appointment persistence integration in progress
FR-18	The system prevents conflicting appointments for the same animal and time slot.	Partial	US-07 / Appointment validation	Database constraint/schema designed; API conflict handling and end-to-end verification still in progress
FR-19	The Administrator can view and update appointment requests.	Partial	US-07 / Admin appointments	Admin UI partially works; backend confirm/cancel/complete persistence in progress
FR-20	The Administrator can complete an approved adoption.	Complete	US-07 / Adoption workflow	UI completion flow works
FR-21	The system maintains adoption history.	Partial	US-07 / Adoption history	UI history exists; live database-backed history integration in progress

FR-22	The system enforces role-based access for adopter and Administrator functions.	Partial	Authentication / Authorization	Client role guard works; server-side role/ownership enforcement integration in progress

5.2 Nonfunctional Requirement Status

ID	Quality Attribute / NFR	Target	Current Evidence / Plan
NFR-01	Usability	Animal profile within three clicks of the homepage; clear validation messages	Home > Browse Animals > View Details is two clicks. Forms show field-level messages (T-11). Met in prototype.
NFR-02	Performance	Pages and search/filter respond within about 2 seconds	Front-end build performance has been checked. Backend/database setup is complete, but API response timing will be measured after feature endpoints are integrated.
NFR-03	Security / Privacy	Hashed passwords; protected routes by role; adopters see only their own records	Database schema stores password_hash and backend dependencies/configuration are in place. JWT, bcrypt, role checks and ownership checks are being integrated into the feature APIs; end-to-end security verification is pending.
NFR-04	Reliability	Reject invalid transitions, duplicates, conflicts	MySQL schema includes key integrity constraints such as unique email and appointment/adoption uniqueness rules. Service-level validation and API error handling are still being connected and tested.
NFR-05	Compatibility	Current Chrome and Edge at desktop sizes	Verified in Chromium at 1366 x 850. Edge check pending.

6. Use-Case Refinement and Object Interaction Modeling

This section traces PawConnect's functional requirements to the eight stable use cases used in the project's use-case diagram and testing section. Two active workflows, application submission within UC-02 and administrator review in UC-03 are expanded into scenarios, sequence diagrams, and a consistent design class model. Statuses follow the supplied updated report: the frontend uses sample data; backend/database foundations are complete; feature integration is ongoing. The models specify intended behavior and do not establish implementation or test completion.

6.1 Requirements-to-Use-Case Traceability

FR ID	Verb-Noun Candidate / Actor Goal	Primary Actor	Use Case ID + Name	Coverage / Status
FR-01	Register Account	Visitor	UC-06 Register, Log In and Log Out	Partial - UI works; backend validation integration in progress
FR-02	Log In / Log Out	Visitor, Adopter, Administrator	UC-06 Register, Log In and Log Out	Partial - UI works; API/DB integration in progress
FR-03	Update Profile	Registered Adopter	UC-08 Manage Adopter Profile	Not yet implemented
FR-04	Browse Animals	Visitor, Adopter	UC-01 Browse, Search and Filter Animals	Partial - UI works; API/DB integration in progress
FR-05	Search / Filter Animals	Visitor, Adopter	UC-01	Partial (no age filter)

FR-06	View Animal Profile	Visitor, Adopter	UC-01	Partial - UI works; API/DB integration in progress
FR-07	Add Animal	Administrator	UC-07 Manage Animals and Medical Records	Partial
FR-08	Update Animal / Status	Administrator	UC-07	Partial
FR-09	Record Medical Care	Administrator	UC-07	Partial
FR-10	Submit Application	Registered Adopter	UC-02 Submit and Track Adoption Application	Partial - UI works; API/DB integration in progress
FR-11	Block Application for Adopted Animal	Registered Adopter	UC-02 (alternate flow)	Partial - server rule integration in progress
FR-12	Track Application	Registered Adopter	UC-02	Partial - UI works; API/DB integration in progress
FR-13	View Applications	Administrator	UC-03 Review Adoption Application	Partial - UI works; API/DB integration in progress
FR-14	Approve / Reject Application	Administrator	UC-03	Partial - UI works; API/DB integration in progress
FR-15	Maintain Animal Status	System on behalf of Administrator	UC-02, UC-03, UC-05, UC-07 (side effects)	Partial
FR-16	View Adoption Record	Administrator	UC-05 Complete Adoption and Maintain History	Partial
FR-17	Request Appointment	Registered Adopter	UC-04 Request and Manage Appointment	Partial
FR-18	Prevent Appointment Conflict	Registered Adopter	UC-04 (alternate flow)	Partial - DB rule designed; API enforcement in progress
FR-19	Manage Appointments	Administrator	UC-04	Partial
FR-20	Complete Adoption	Administrator	UC-05	Partial - UI works; API/DB integration in progress
FR-21	Maintain Adoption History	Administrator	UC-05	Partial - UI works; API/DB integration in progress
FR-22	Enforce Role-Based Access	All actors	Cross-cutting rule of every use case (UC-06 logs in)	Partial - client guard works; server enforcement in progress

- Each selected use case is a business process started by an actor that gives that actor a useful result (for example, UC-02 gives the adopter a pending application they can track). Low-level actions such as "validate form" or "update animal status" are steps inside use cases, not use cases.
- Every functional requirement maps to at least one use case. Gaps: no use case is unneeded; UC-08 (FR-03) and the alternate flows for FR-11 and FR-18 are not implemented yet.

6.2 Updated Use Case Model

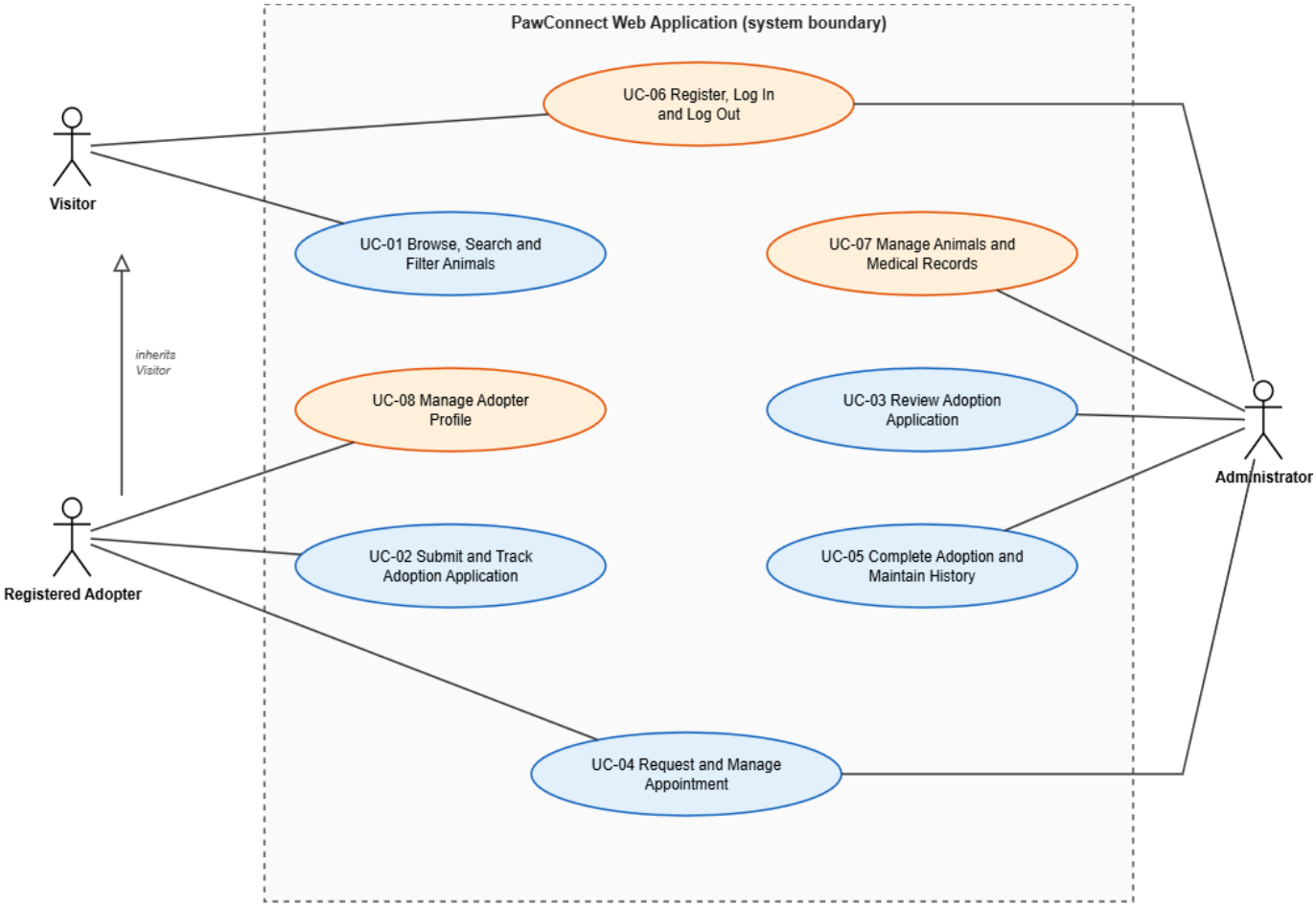


Figure 6.1 PawConnect business-level use case model

Refinement	Earlier Baseline	Current Model	Reason
UC-01–UC-05	Original five use cases, as described in the updated report.	Retained with stable IDs.	Preserves traceability across proposal, models, and tests.
UC-06	Not present in the earlier model, according to the updated report.	Register, Log In and Log Out.	Covers FR-01 and FR-02.
UC-07	Not present in the earlier model, according to the updated report.	Manage Animals and Medical Records.	Covers FR-07–09 and associated state rules.
UC-08	Not present in the earlier model, according to the updated report.	Manage Adopter Profile.	Covers FR-03; implementation has not started.
UC-02 / UC-04 / UC-05	Existing application, appointment, and adoption goals.	Track applications; schedule after approval; complete adoption and retrieve history.	Clarifies grouped flows without renumbering.

6.3 Expanded Use Cases for Core Workflows

Field	Use Case A	Use Case B
ID and name	UC-02 Submit and Track Adoption Application	UC-03 Review Adoption Application
Primary actor	Registered Adopter	Administrator
Related requirements	FR-10–12, FR-15, FR-22; US-05	FR-13, FR-14, FR-15, FR-22; US-06
TUCBW	This use case begins with the adopter choosing to apply from an animal profile.	This use case begins with the administrator opening the application review area.
TUCEW	This use case ends with the adopter receiving the saved application reference and Pending status.	This use case ends with the administrator receiving confirmation of the saved approval or rejection.
Preconditions	The adopter is logged in and the selected animal exists.	The administrator is logged in; a Pending application exists.
Success postconditions	A Pending application is persisted and associated with the adopter and animal.	The decision and reviewer are persisted; related animal state remains consistent.
Implementation status	Partial overall; frontend prototype exists, backend integration remains in progress.	Partial overall; frontend prototype exists, backend integration remains in progress.

Use Case A - UC-02 Submit and Track Adoption Application (steps)

Step	Actor Action	System Response
1	The adopter opens an animal profile and chooses "Apply for Adoption".	
2		The system checks that the animal is accepting applications and shows the application form with the adopter's name and email.*
3	The adopter enters reason, pet experience, living arrangement, yard and other pets, employment status and references, then submits.	
4		The system validates the form, confirms the animal is not Adopted and the adopter has no active application for it, saves a Pending application, sets the animal to Application Pending, records the status change, and shows a confirmation.*
5	The adopter chooses "View My Applications".	
6		The system retrieves and displays only that adopter's applications with status and any appointment.*
Alt 4a		A required field is empty: the system shows field messages and saves nothing.
Alt 4b		The animal is Adopted or the adopter already has an active application: the system rejects the request with a clear message.
Alt 6a	The adopter withdraws a pending application.	The system marks it Withdrawn and, if no other pending application exists, returns the animal to

		Available.*

Use Case B - UC-03 Review Adoption Application (steps)

Step	Actor Action	System Response
1	The administrator opens Applications.	
2		The system retrieves all applications with adopter and animal details and shows them with status filters.*
3	The administrator expands a Pending application to read the answers.	The system displays the full application.
4	The administrator chooses Approve or Reject and confirms in the dialog.	
5		The system verifies the application is Pending, records the decision (administrator, time, note), updates the animal status (Approved, or Available if rejected and no other pending application), logs the status change, and refreshes the list.*
6	The administrator sees the new status.	
Alt 5a		The application is no longer Pending: the system rejects the decision and shows a message.

Nontrivial system-response steps

Marked Step	Why background processing is required
UC-02 step 2	Retrieve the animal record and check its status (eligibility) before showing the form; data-dependent response.
UC-02 step 4 (selected for 6.4-6.5)	Validate input, retrieve the animal, query for a duplicate active application, save the application, update the animal status, and write a status-history row; several objects and database writes.
UC-02 step 6	Retrieve only the logged-in adopter's applications (ownership filter) and join appointment data.
UC-02 step Alt 6a	Update the application and conditionally revert the animal status after checking other pending applications.
UC-03 step 2	Retrieve all applications and join adopter and animal details.
UC-03 step 5 (selected for SD-02)	Check current state, save the decision with audit fields, apply a valid animal-status transition, and log it; must be consistent across tables.

6.4 Object-Interaction Scenario for a Selected Nontrivial Step

Selected Item	Team Response
Use case and step	UC-02, A4; triggered by A3.
Why nontrivial	Retrieve authenticated adopter and current animal, validate data and eligibility, detect a duplicate, create and save the application, and produce a persisted-data confirmation.
Participating objects	Adopter; ApplicationPage boundary; ApplicationController control; PawConnectRepository persistence abstraction; AdoptionApplication entity.
Scenario start	The adopter submits the completed form.
Scenario end	ApplicationPage displays the saved reference and Pending status.

1. The adopter submits the animal identifier and completed form data to ApplicationPage.

2. ApplicationPage requests submission processing from ApplicationController with the session, animal identifier, and form data.

3. ApplicationController retrieves the authenticated adopter through PawConnectRepository using the session.

4. ApplicationController validates the required form fields within its validation operation.

5. ApplicationController retrieves the selected animal through PawConnectRepository using the animal identifier.

6. ApplicationController checks the retrieved animal status within its eligibility operation.

7. ApplicationController retrieves any existing active application through PawConnectRepository using the adopter and animal identifiers.

8. ApplicationController checks the retrieved application result within its duplicate validation operation.

9. ApplicationController creates an AdoptionApplication with the adopter identifier, animal identifier, and validated form data.

10. ApplicationController saves the Pending application through PawConnectRepository and obtains the persisted record.

11. ApplicationController sends the saved application identifier and status to ApplicationPage.

12. ApplicationPage displays the application identifier and Pending status to the adopter.

Continuity check: after the first sentence, every subject is either the object acted upon in the preceding sentence or a subject used earlier. The controller remains the initiating subject for retrieval and validation operations. The scenario describes the normal success path; alternatives are defined in Section 6.3.

6.5 Scenario Table and Sequence Diagram

Line #	Subject	Subject Action	Other Data / Object(s)	Object Acted Upon
1	Adopter	submits	completed application form data	ApplicationFormPage
2	ApplicationFormPage	validates	required fields are not empty	itself (form data)
3	ApplicationFormPage	sends	application request (animalId, form data) with JWT cookie	ApplicationController
4	ApplicationController	authenticates	JWT cookie and the adopter	itself (AuthMiddleware

			role	checks)
5	ApplicationController	asks	to submit the application (userId, animalId, form data)	ApplicationService
6	ApplicationService	requests	the animal record by animalId	AnimalRepository
7	AnimalRepository	returns	the animal with its current status	ApplicationService
8	ApplicationService	asks	whether the adopter has an active application for the animal	ApplicationRepository
9	ApplicationRepository	returns	the result (none found)	ApplicationService
10	ApplicationService	asks	to create a Pending application	ApplicationRepository
11	ApplicationRepository	returns	the saved application	ApplicationService
12	ApplicationService	asks	to move the animal to Application Pending	AnimalStatusService
13	AnimalStatusService	asks	to update the status and add a status-history row	AnimalRepository
14	AnimalRepository	confirms	the update	AnimalStatusService
15	AnimalStatusService	confirms	the transition	ApplicationService
16	ApplicationService	returns	the Pending application	ApplicationController
17	ApplicationController	responds	201 Created with the application	ApplicationFormPage
18	ApplicationFormPage	displays	"Application Submitted" with status Pending	Adopter

Retrieval messages use result = operation(parameters) to define return data. Solid call arrows carry those operation signatures; separate dashed return arrows are omitted. The repository is an abstraction over database access rather than a second domain entity. savePendingApplication persists the application and any required animal-state change together; it returns only after a successful commit.

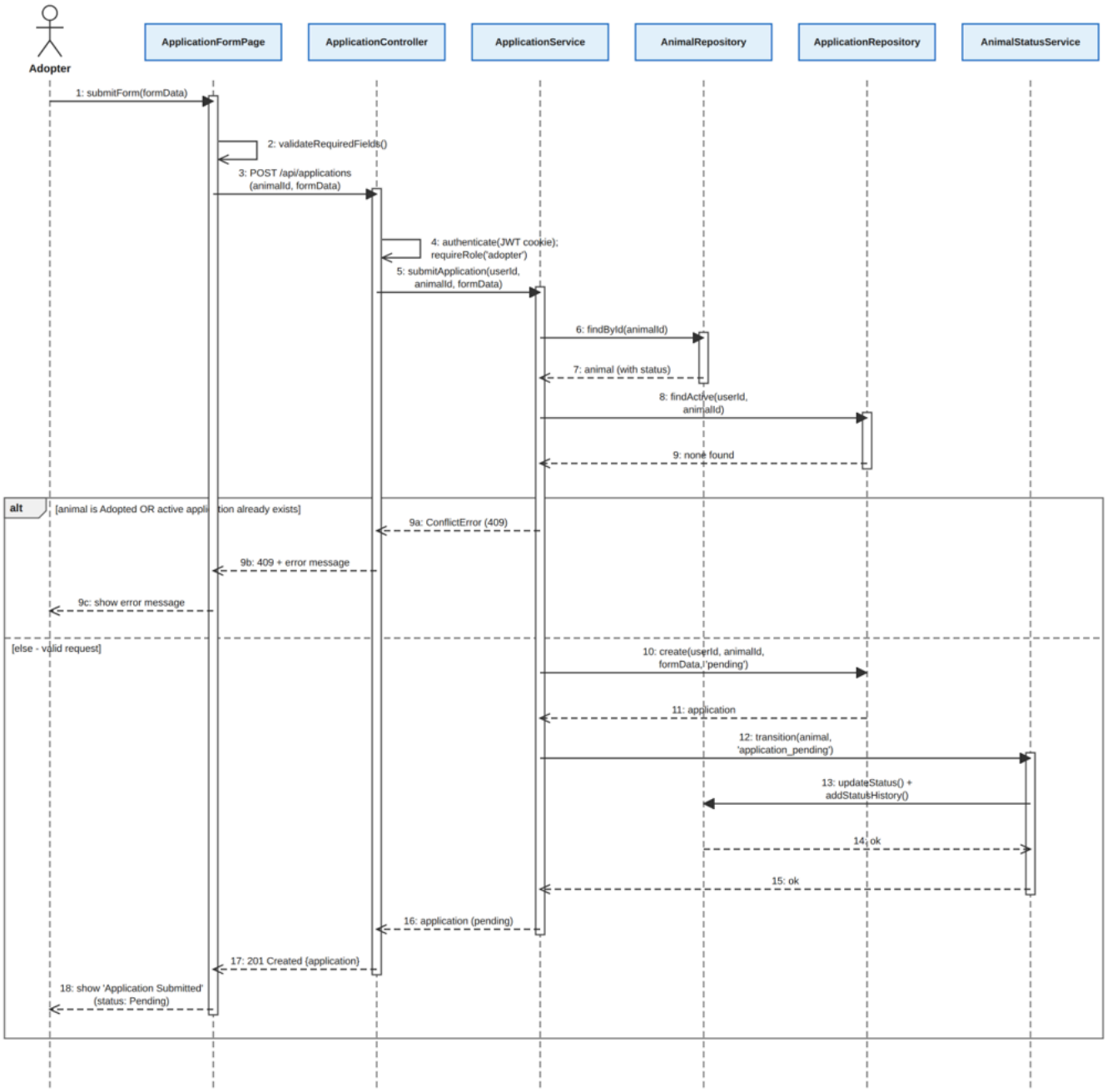


Figure 6.2 UC-02 application submission sequence

6.6 Second Sequence Diagram - Another Core Use Case

UC-03 Review Adoption Application, nontrivial step B6, triggered by the administrator action B5. The administrator has already opened a Pending application. The controller reloads authoritative records, validates authorization and the decision, coordinates the application update and transaction, and displays the persisted result.

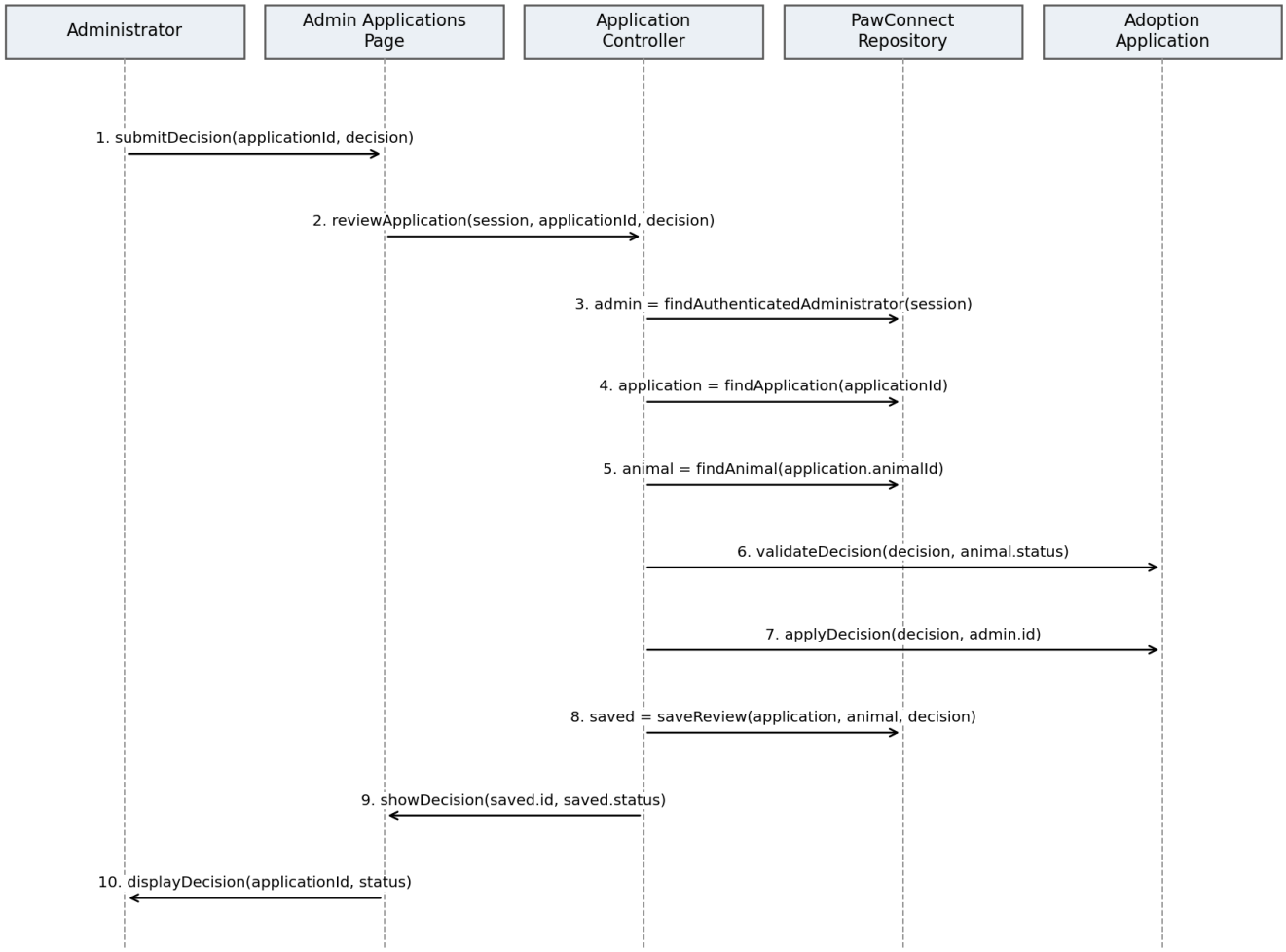
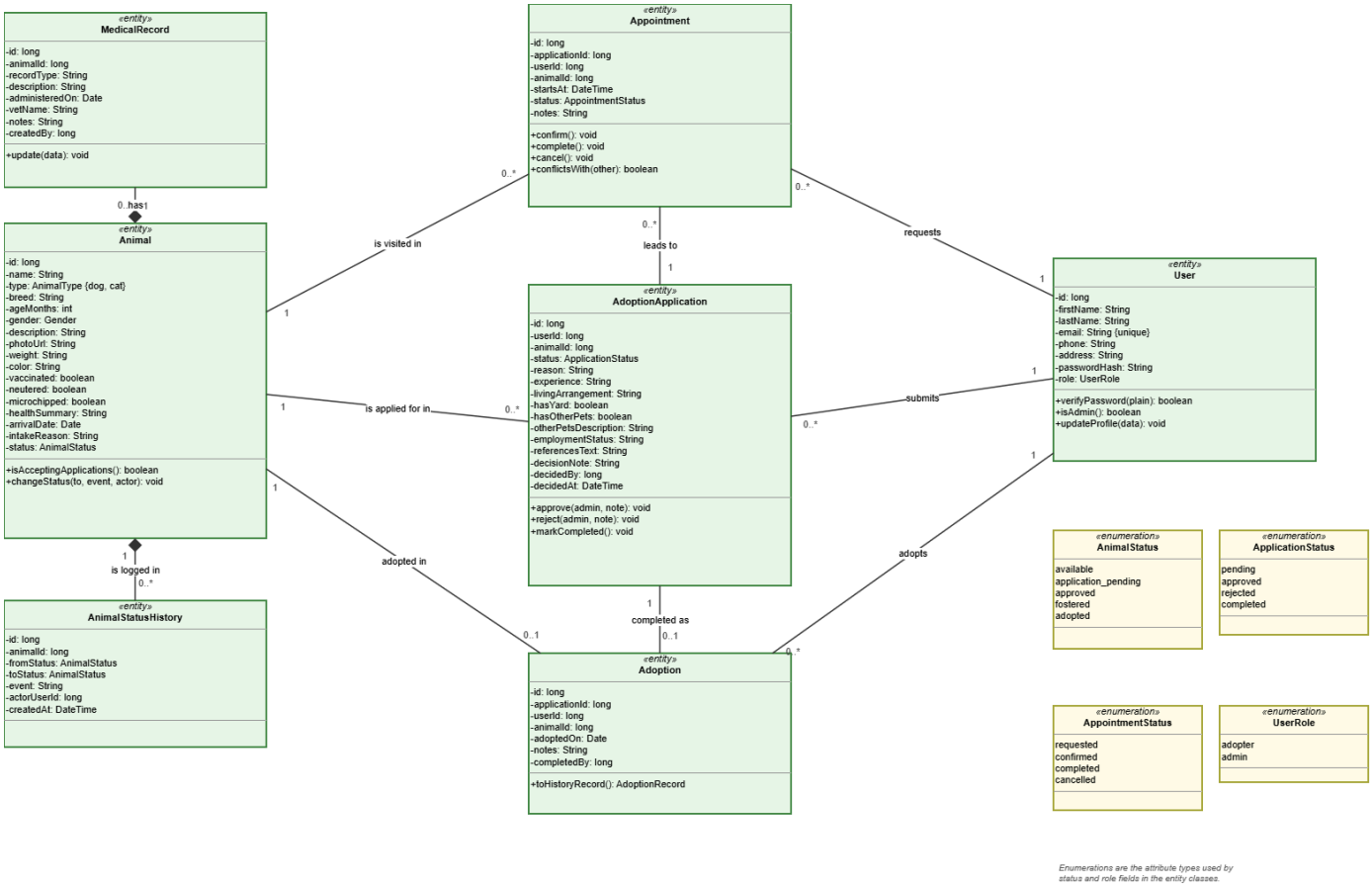


Figure 6.3 UC-03 review decision sequence for the normal success path

saveReview checks that the application remains Pending and saves the decision, reviewer, and required animal-state change in one transaction. Concurrent or invalid transitions fail without partial updates. The repository returns the persisted record before the page receives a success confirmation. Error paths follow the alternatives in Section 6.3; no optional fragments are required for these normal-path diagrams.

6.7 Current Design Class Diagram and Model Consistency



Domain associations: One User can submit multiple AdoptionApplications. Each application belongs to one adopter and one animal. An Animal can have multiple applications, medical records, status-history entries, and appointments. Each Adoption links one adopter, one animal, and one application; each animal and application can have at most one completed adoption in the current schema. Administrator references record who reviews applications and performs status changes. Composition represents Animal's ownership of MedicalRecord and AnimalStatusHistory.

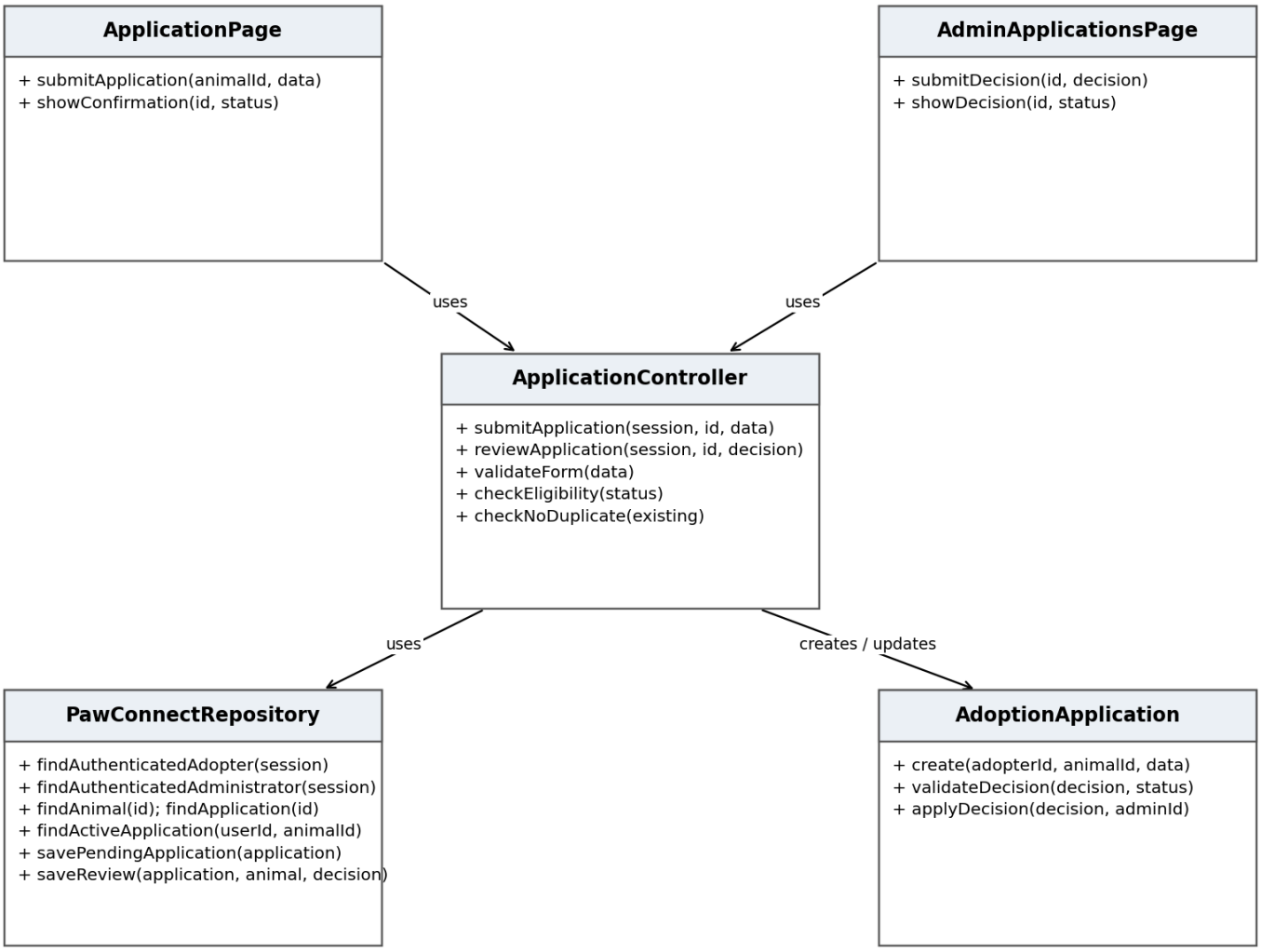


Figure 6.4c Boundary control and persistence classes discovered from the two sequence diagrams

Interaction Responsibility	Class or Operation	Consistency Check
Submission and review requests	ApplicationPage / AdminApplicationsPage	Boundary operations match actor-triggered sequence messages.
Identity and access checks	PawConnectRepository.findAuthenticatedAdopter / findAuthenticatedAdministrator	Each operation validates the supplied session and role before returning identity.
Submission processing	ApplicationController.submitApplication, validateForm, checkEligibility, checkNoDuplicate	Uses only values supplied by the page or retrieved from persistence.
Application state changes	AdoptionApplication.create, validateDecision, applyDecision	Pending creation and review decisions map to entity responsibilities.
Persistence	PawConnectRepository.findAnimal, findApplication, findActiveApplication, savePendingApplication, saveReview	Retrieved objects are saved after mutation; related updates commit together.
Confirmation	ApplicationPage.showConfirmation / AdminApplicationsPage.showDecision	Success is shown only after repository save returns.

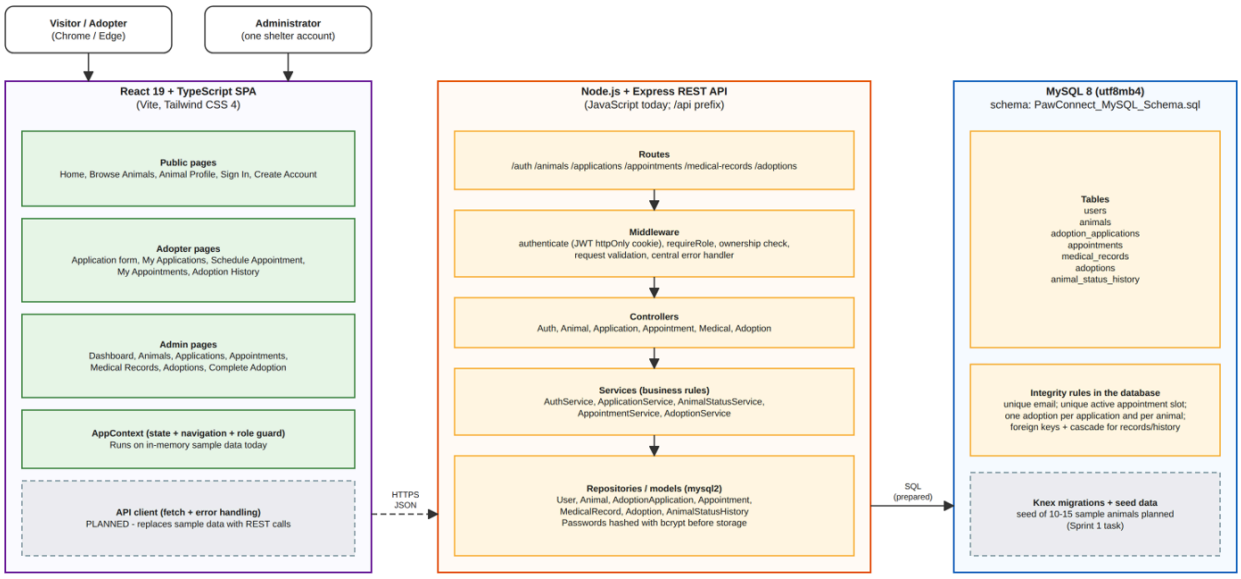
6.8 Modeling Evolution Summary

Modeling Element	Earlier Description	Refined Model	Reason
Use cases and actors	Broad adoption workflow in the executive summary.	UC-02 Submit and Track Adoption Application and UC-03 Review Adoption Application retain their stable IDs and explicit boundaries.	Keeps actor responsibilities and outcomes clear.
Interaction responsibilities	The system validates and processes an application.	Boundary captures input; controller coordinates; entity changes state; repository persists.	Makes each nontrivial system step explainable and traceable.
Class relationships	Animals, users, applications, appointments, and history are in scope.	Explicit references, multiplicities, reviewer association, and transaction responsibilities.	Supports persistence and avoids inconsistent application/animal states.
Validation and status	FR-11, FR-15, FR-22 and NFR-04 define constraints.	Eligibility, authorization, duplicate, and stale-decision checks appear in the expanded flows.	Links quality and business rules to concrete interactions.
Implementation alignment	Section 5 marks the selected workflows Partial.	Models describe active workflows and proposed design, with no additional completion claims.	Actual code and proposal comparison remain evidence checks.

7. Architecture, Data, and UI Progress

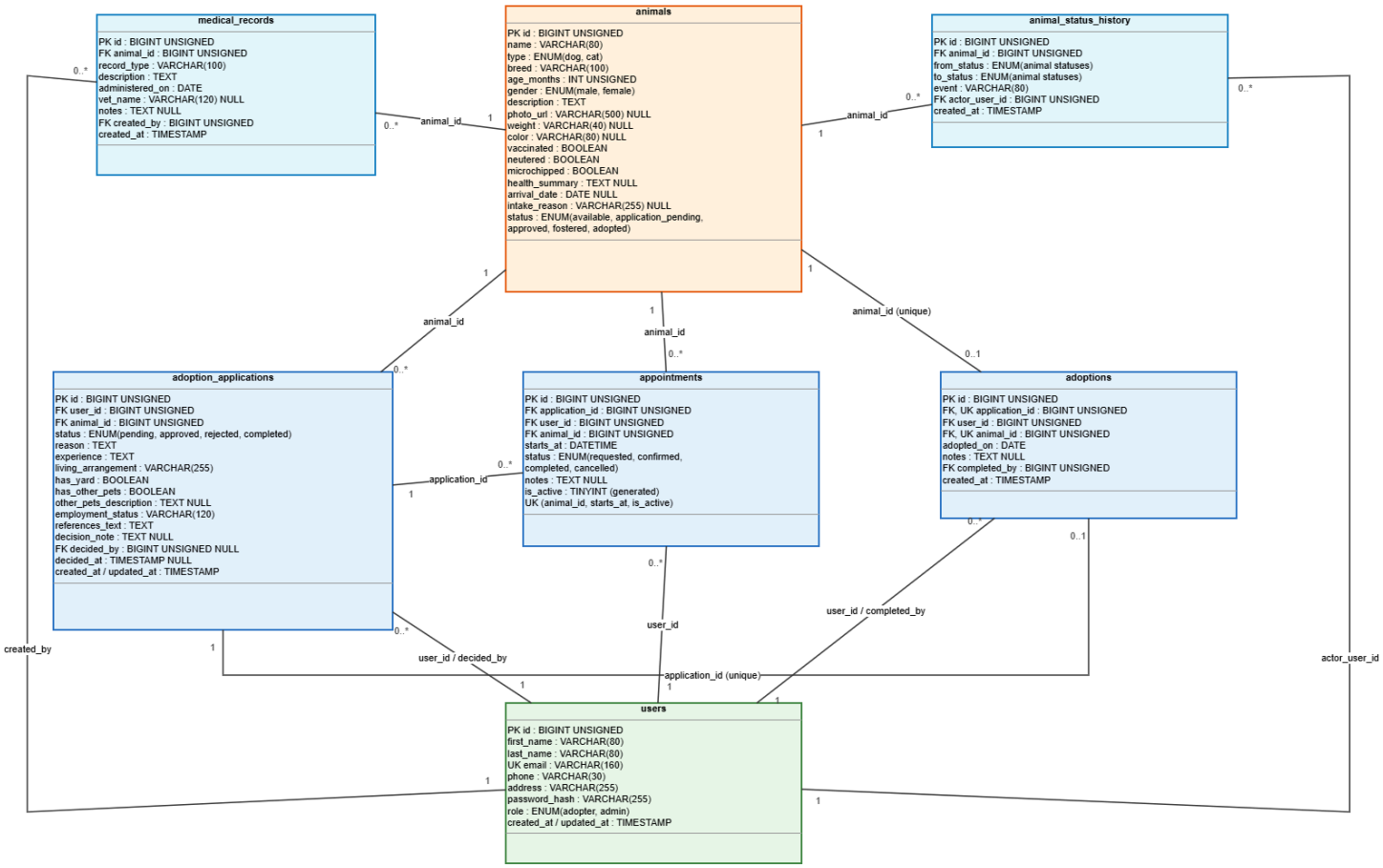
7.1 Current Architecture / Component View

PawConnect is a three-tier web application: a React single-page app, a Node.js/Express REST API, and a MySQL database. The frontend prototype is implemented, and the backend/database foundation is complete: the Express project structure, middleware/configuration setup, database connection, MySQL schema, and ERD are in place. Feature-level controllers/services/repositories and the frontend API client are still being completed and integrated. Compared with the proposal, the architectural layers are unchanged; the technology choices and internal backend structure have been refined.



7.2 Database / Persistence Model

PawConnect Database ERD (MySQL 8) - generated from PawConnect_MySQL_Schema.sql



Domain object	Stored in	How it is stored or retrieved
Animal	animals	Listing and search use the (type, status) and name indexes; status is an ENUM of the five lifecycle values.
AdoptionApplication	adoption_applications	Created as pending (SD-01); decision fields decision_note, decided_by, decided_at are set on review (SD-02); lookups by (user_id, animal_id) and (animal_id, status).
AnimalStatusHistory	animal_status_history	One row per transition with from_status, to_status, event and actor; written by AnimalStatusService through AnimalRepository.
Appointment	appointments	Unique key (animal_id, starts_at, is_active) with a generated is_active column allows only one Requested/Confirmed booking per animal and time (FR-18).
Adoption	adoptions	Unique application_id and unique animal_id guarantee one adoption per application and per animal (SD-03).
MedicalRecord	medical_records	Rows cascade-delete with the animal; created_by records the admin.
User	users	Unique email; password_hash only; role ENUM adopter/admin.

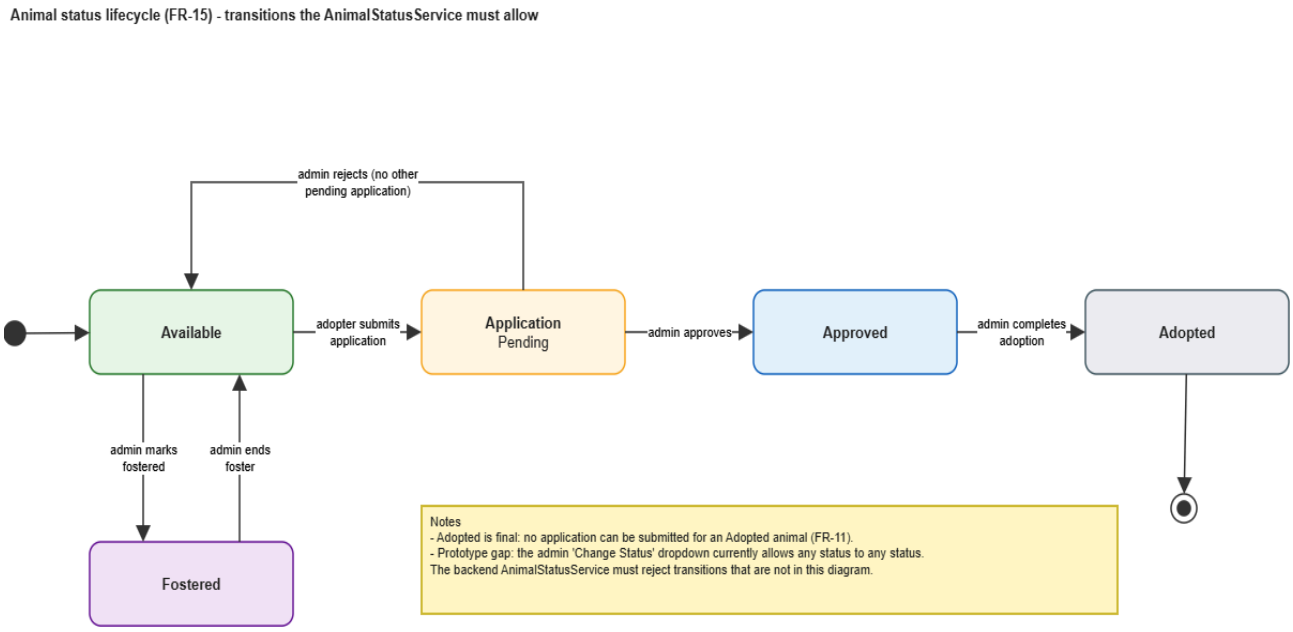


Figure 7.3 - Animal status state diagram

7.3 UI / UX Progress

Implemented screens are shown in Section 4.1 (Figures 4.1 to 4.13) and cover 17 page states: Home, Browse Animals, Animal Profile, Sign In, Create Account, Application Form, My Applications, Schedule Appointment, My Appointments, My Adoption History, and the admin pages Dashboard, Animals, Medical Records, Applications, Appointments, Complete Adoption, and Adoption History. The Figma prototype remains the design reference:

PawConnect Web App Prototype (Figma)

UI changes caused by use-case and workflow refinement:

(1) adopters book an appointment only after their application is Approved;

(2) the admin completes an adoption from a completed appointment through a two-step review/confirm page;

(3) a My Adoption History page was added for adopters;

(4) animal status colors and badges were standardized across listing, profile and admin pages. The navigation paths are summarized in Figure 7.4.

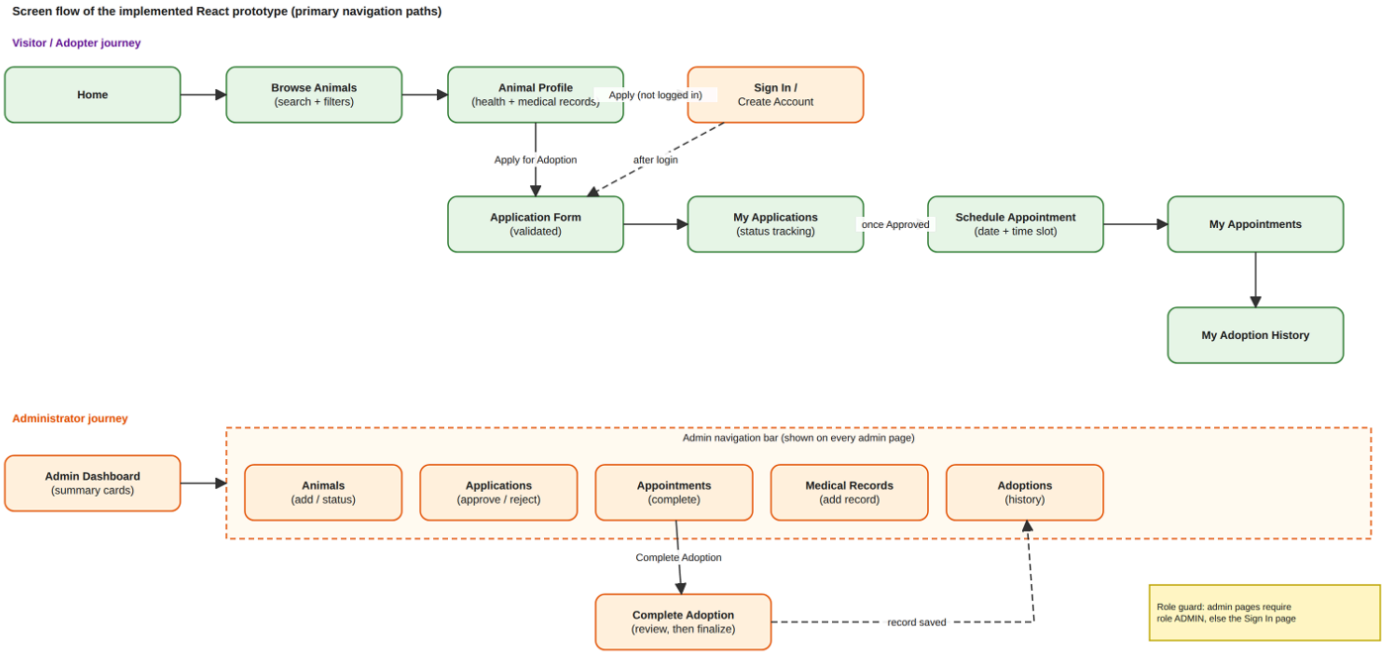


Figure 7.4 - Screen flow of the implemented prototype.

8. Implementation Progress

8.1 Technology Stack and Repository Structure

Area	Current Technology / Structure	Notes
Frontend	React 19 + TypeScript + Vite + Tailwind CSS 4 (frontend/)	14 page components plus shared AppContext (state, navigation, role guard), Navbar and StatusBadge; about 3,500 lines of TypeScript/TSX.
Backend	Node.js + Express (backend/): routes, controllers, services, models, middleware, config	Backend project setup is complete: Express structure, configuration, middleware foundation, database connectivity and dependencies are in place. Feature routes/controllers/services are still being implemented and integrated.
Database	MySQL 8 with Knex migrations and seeds	MySQL 8 database setup is complete with the seven-table schema/ERD and persistence constraints. Migrations/seeds and feature-level repository usage are being finalized.
Auth	JWT in an httpOnly cookie; passwords hashed with bcrypt	Authentication design and backend setup are in place; JWT/bcrypt and role/ownership checks are being integrated into the feature endpoints.
Testing	Vitest + Supertest for backend/API; Playwright for UI	UI testing has been performed; backend/API automated tests are the next step after feature endpoint integration.
Build / Deployment	Vite build; Vercel planned	No deployed URL yet.

8.2 Key Implemented Components / Classes

Component / Class	Responsibility	Related UC / Sequence Diagram	Status
Express API / Server	Initializes the backend, common middleware, error handling, and route	Architecture / Foundation	In progress

	registration.		
Authentication Module	Handles registration, login, logout, password hashing, and current-user authentication.	US-01 / FR-01–FR-02	Partial
Authorization Middleware	Checks authenticated users, administrator role, and ownership before protected operations.	FR-22	Partial
Animal Data Module	Reads animal records used by listings, filters, and detailed profiles.	UC-01	In progress
Application Controller / Service	Validates and processes adoption application requests and application decisions.	UC-02 / SD-01	In progress
Application Repository	Stores and retrieves adoption application records through the shared database layer.	UC-02 / SD-01	In progress
Animal Status Service	Applies valid animal-state transitions and records status history.	UC-02 / UC-03	In progress
MySQL Persistence Layer	Provides relational storage for users, animals, medical records, applications, appointments, adoptions, and status history.	Architecture / Data Model	In progress
Validation / Security Layer	Uses server-side validation, password hashing, JWT cookies, and protected-route checks.	NFR-03 / NFR-04	In progress
Backend Integration Tests	Verifies API behavior, authorization, validation, persistence, and core workflow rules.	FR-01–FR-22 as applicable	Planned / In progress

8.3 Repository Evidence

- Repository URL: https://github.com/hbprasad0110-rgb/PawConnect
- Commit/branch strategy: feature branches (for example feature/frontend-setup, feature/backend) merged into development and then main through pull requests.
- Code review/pull request approach: Code review / pull request approach: main requires at least one approving review from a teammate before merge (stated in the README).

9. Testing Progress and Quality Evidence

9.1 Test Strategy to Date

- **Unit testing:** Node.js's built-in test runner checks animal-status transitions, including adoption, rejection, withdrawal, and invalid transitions.
- **API testing:** Supertest checks the /api/health endpoint. This test does not require a database connection and does not verify database-backed feature APIs.
- **Static and build checks:** TypeScript checks the frontend for type errors. Backend and frontend production builds verify that both applications compile successfully.
- **UI and database testing:** Browser workflows and MySQL integration were not tested. Registration, login, application persistence, appointment conflicts, and complete adoption workflows remain pending verification.

9.2 Executed Test Case

Test ID	Related Requirement	Test Description	Expected Result	Actual Result	Result
T-01	FR-15	Check the adoption status progression.	Available → Application Pending → Approved → Adopted.	All assertions passed.	Pass
T-02	FR-15	Check status after application rejection.	Animal returns to Available when no other active application exists; otherwise remains Application Pending.	All assertions passed.	Pass
T-03	FR-15	Check status after application withdrawal.	Animal moves to the appropriate state based on its current state and other active applications.	All assertions passed.	Pass
T-04	FR-15	Attempt an invalid transition from Adopted to Available.	Operation throws an error.	An expected error was thrown.	Pass
T-05	API foundation	Request GET /api/health.	HTTP 200 with status ok.	Expected response received.	Pass

Automated test summary: Five test cases passed, with zero failures. These tests contain multiple assertions and provide limited coverage of the application.

Check ID	Verification	Command	Actual Result
C-01	Frontend type checking	npm run typecheck --prefix frontend	Passed; exit code 0.
C-02	Backend and frontend production builds	npm run build	Passed; exit code 0. Frontend bundle-size warning reported.

Evidence references:

- E01_automated_tests.txt - automated test output.
- E02_frontend_typecheck.txt - TypeScript check output.
- E03_production_build.txt - build output and warning.
- test_scripts/ - existing repository test files.
- execution_metadata.json - execution timestamps, environment, and tested commit.

These files are included in PawConnect_Test_Evidence.zip

9.3 Defects / Issues

Issue ID	Description	Severity	Status	Resolution / Plan
BUG-01	Register accepts an email that already exists (T-02); FR-01 requires duplicate rejection.	High	Open	Enforce unique email in the API (users_email_unique) and show a clear message; add API test.
BUG-02	No appointment conflict check (T-17); the same animal, date and time can be booked twice.	High	Open	Connect appointments_unique_active_slot and return a conflict error; add test.
BUG-03	Admin "Change Status" dropdown allows any status to any status, bypassing FR-15 rules.	Medium	Open	Route all changes through AnimalStatusService.assertValidTransition (Figure 7.3).
BUG-04	Sample data is inconsistent: Buddy is Available while having an Approved application and a scheduled appointment; Mochi is Available with a Pending application.	Low	Open	Correct seed data so statuses follow the lifecycle.
BUG-05	Status vocabularies differ: prototype uses Withdrawn applications and Scheduled appointments; schema uses Completed applications and Requested/Confirmed appointments.	Medium	Open	Decide the final lists in Sprint 3 and update SRS, schema, types and UI together.
BUG-06	Backend/database setup is complete, but the frontend is not yet connected to the REST API and feature-level endpoint integration is still in progress.	Medium	Open	Complete feature controllers/services/repositories, connect the frontend API client, add migrations/seeds to the final branch if needed, and update README run steps.
BUG-07	Application confirmation says the adopter will receive an email, but notifications are out of scope.	Low	Open	Change the wording to refer to My Applications.
BUG-08	Demo passwords are stored as plain text in sample data and compared in the browser.	Medium	Open	Acceptable only for the prototype; replaced by bcrypt hashing and server login.
BUG-09	Adoption application form (KAN-35): validation and form submission are not working as expected (FR-10, FR-11). Some form components also do not display properly while scrolling.	High	Open	Reproduce and fix the required-field validation and the submit handler; fix the form layout so every field and the Submit button stay usable while scrolling; connect submit to the applications API once the integration is done; add a test for valid and invalid submissions.

10. Risks, Impediments, and Corrective Actions

Impediment / Risk	Effect on Project	Action Taken	Owner	Status
Backend/database foundation complete, but frontend/API integration is still in progress	Stories are not yet fully verified end to end; integration issues may appear late	Use Sprint 3 to complete feature APIs, connect the frontend API client, and run daily integration checks	Harsha Prasad, Himanshu Shimpi	Mitigating
Prototype and schema vocabularies differ (BUG-05)	Rework in UI, types and SQL if decided late	Decide the final status lists at Sprint 3 planning and update all artifacts together	Harsha Prasad (Product Owner)	Open
Large scope for three people (22 FRs)	Lower-priority items may slip	Protect the Must flow (browse > apply > review > adopt); defer the age filter and optional polish if needed	Harsha Prasad	Mitigating
Uneven visible contribution across members	Grading and workload fairness	Assign backend ownership by module with pull requests per member; record work in Jira and commits	Dipanjana Das (Technical Lead), all	Open
Backend automated tests are not complete	Regressions may appear as feature APIs are integrated	Add Vitest/Supertest coverage during API integration and keep Playwright as the UI smoke test	Himanshu Shimpi (Scrum Master)	Open
Two tracking tools (Jira and GitHub)	Status can drift	Jira is the source of truth; link pull requests to Jira keys	Himanshu Shimpi	Mitigating

11. Team Contribution and Collaboration Evidence

Team Member	Completed Stories / Tasks	Modeling / Documentation	Implementation	Testing / Review	Evidence
Harsha Prasad (Product Owner + Developer)	Repository/project setup; Jira backlog and sprint board; Jira-GitHub traceability (KAN-9); adopter registration (KAN-1); login/logout (KAN-2); initial animal data model (KAN-3); adopter profile view/update (KAN-11); role-based access check (KAN-12); registration/login integration (KAN-15); animal listing API integration (KAN-16); adoption application integration (KAN-17); appointment workflow integration (KAN-22); animal management integration (KAN-23); adoption completion & history integration (KAN-27); admin dashboard summary integration (KAN-28).	Requirements baseline; Updated Use Case Model and UML Use Case Diagram; UC-02 object-interaction scenario; scenario table and sequence diagram; Current Design Class Diagram and model-consistency check; backend architecture; ERD/database persistence model; Technology Stack and Repository Structure; Key Implemented Components / Classes; Repository Evidence; Implementation Progress documentation; functional and nonfunctional requirement status updates.	Node.js/Express REST API; MySQL integration through Knex; JWT authentication using HTTP-only cookies; role-based authorization; Zod validation; animal, medical-record, application, appointment, adoption, status-history and admin-summary APIs; frontend-backend-database integration for the core adoption workflow.	Backend/API verification; database persistence checks; authentication and role-access verification; end-to-end workflow verification; validation/error-handling checks; PR review and merge. Code/PR review performed by Himanshu Shimpi. Integration and sprint-testing support provided to the team.	Jira KAN board and tickets KAN-1, KAN-2, KAN-3, KAN-9, KAN-11, KAN-12, KAN-15, KAN-16, KAN-17, KAN-22, KAN-23, KAN-27, KAN-28; GitHub commits and pull requests; teammate review by Himanshu Shimpi; repository/branch evidence; MySQL/database screenshots; ERD; Updated Use Case Diagram; Design Class Diagram; sequence diagram; architecture diagram; sprint-board screenshots.

Himanshu Shimpi (Scrum Master + Developer)	React/TypeScript frontend for US-01 to US-07 screens; frontend API integration (KAN-14); animal browsing and profile UI (KAN-33, in review); frontend authentication and session flow (KAN-32, in progress)	README; project docs; sprint review and retrospective notes (KAN-13, to follow)	React + TypeScript frontend started from the Figma design and wired to the backend API; frontend; README and .gitignore commits; browsing, animal profile, register and login screens connected to the API; login and session handling (KAN-32, in progress)	Integration and sprint testing of the register, login, browse and profile flow (KAN-6, in review); UI checks on the browsing and profile screens (KAN-33); cross-browser testing on Chrome, Edge and Firefox (KAN-31, planned); peer review of Harsha's login and registration integration (KAN-15)	Jira tickets KAN-6, KAN-14, KAN-31, KAN-32, KAN-33; Frontend screenshots refer section 4
Dipanjana Das (Technical Lead + Developer)	Repository and project setup (KAN-7); GitHub branch strategy and protection (KAN-8); backend/API testing (KAN-20, in review); automated API and integration tests (KAN-29, in progress); end-to-end workflow testing (KAN-21, in progress)	Branch strategy and pull request workflow notes (KAN-8); test plan for the API and end-to-end tests (planned); final documentation and evidence (KAN-26, planned for the wrap-up phase)	Repository structure and project setup; branch protection on main; automated API and integration test suite (KAN-29, in progress); deployment and demo environment (KAN-24, planned); next: base admin dashboard (KAN-5) and animal detail page (KAN-10)	Backend/API tests (KAN-20, in review); automated integration tests (KAN-29, in progress); end-to-end testing of the browse, apply, approve, appointment and adoption flow (KAN-21, in progress); peer review of Himanshu's frontend pull requests	Jira tickets KAN-7, KAN-8 (done); KAN-20, KAN-21, KAN-29; test run results, test result uploaded in git

12. Plan for Remaining Sprints

Remaining Sprint / Period	Goal	Planned Stories / Use Cases	Major Design/Test Work	Major Risk
Next Sprint (Sprint 3) (Oct. 19 – Oct 31, 2026)	Complete feature APIs and connect the existing frontend to the backend/database for the core flow	US-01 to US-06; UC-06, UC-01, UC-02, UC-03, UC-07; BUG-01, BUG-06	Use completed backend/database foundation; implement remaining auth, animal, application and admin endpoints; connect frontend API client; seed data; run Supertest integration tests; align status vocabularies	Integration defects or endpoint/business-rule gaps
Following Sprint (Sprint 4) (Nov. 1 – Nov. 15, 2026)	Appointments, adoption completion, history, role enforcement	US-07; UC-04, UC-05, UC-08; BUG-02, BUG-03	Finish AppointmentService, AdoptionService and AnimalStatusService rules/transactions; complete adopter profile; verify FR-16 to FR-22 with API/database tests	Transaction and conflict edge cases
Finalization (Nov. 16 – Final Submission Date)	Testing, deployment, documentation, presentation	All Must requirements verified; remaining defects	Full regression (UI and API), Chrome and Edge checks, performance timing, Vercel deployment, final UML update, COCOMO comparison with actual size	Late defects, deployment issues

13. Links and Supporting Evidence

- Repository URL: https://github.com/hbprasad0110-rgb/PawConnect
- Project board/backlog URL: Backlog URL
- Current deployed/demo URL: Not deployed yet, the application currently runs locally for development and testing. Public deployment is planned before the final submission.

14. Progress Report Submission Checklist

☐Executive progress summary is complete	☐A five-column scenario table is included for the selected scenario
☐Scope and requirement changes are documented	☐At least 2 sequence diagrams represent important implemented/active workflows
☐Sprint history and Agile evidence are included	☐Sequence diagrams are consistent with scenarios and the current design class diagram
☐A working partial application is demonstrated	☐Architecture/data/UI progress is documented
☐Updated SRS / requirement status is included	☐Implementation evidence and repository links are included
☐Requirements are traced to business-level use cases with stable IDs	☐Testing evidence and defect status are included
☐Updated UML use-case diagram is included	☐Risks/impediments and remaining plan are documented
☐At least 2 core use cases are expanded with TUCBW/TUCEW and actor-system steps	☐Individual contribution evidence is complete
☐Nontrivial system-response steps are identified and justified	☐No unauthorized AI-generated content is included
☐One selected nontrivial step is developed into an object-interaction scenario	